

คู่มือสำหรับโปรแกรมเมอร์

Embedded Web Service Development Programmer Manual

โดย

นายสุวิทย์ มัชฌิม 42010411

นายเอกกร อินนอ 42010452

อาจารย์ที่ปรึกษา

ผศ.อภิเนตร อุนากุล

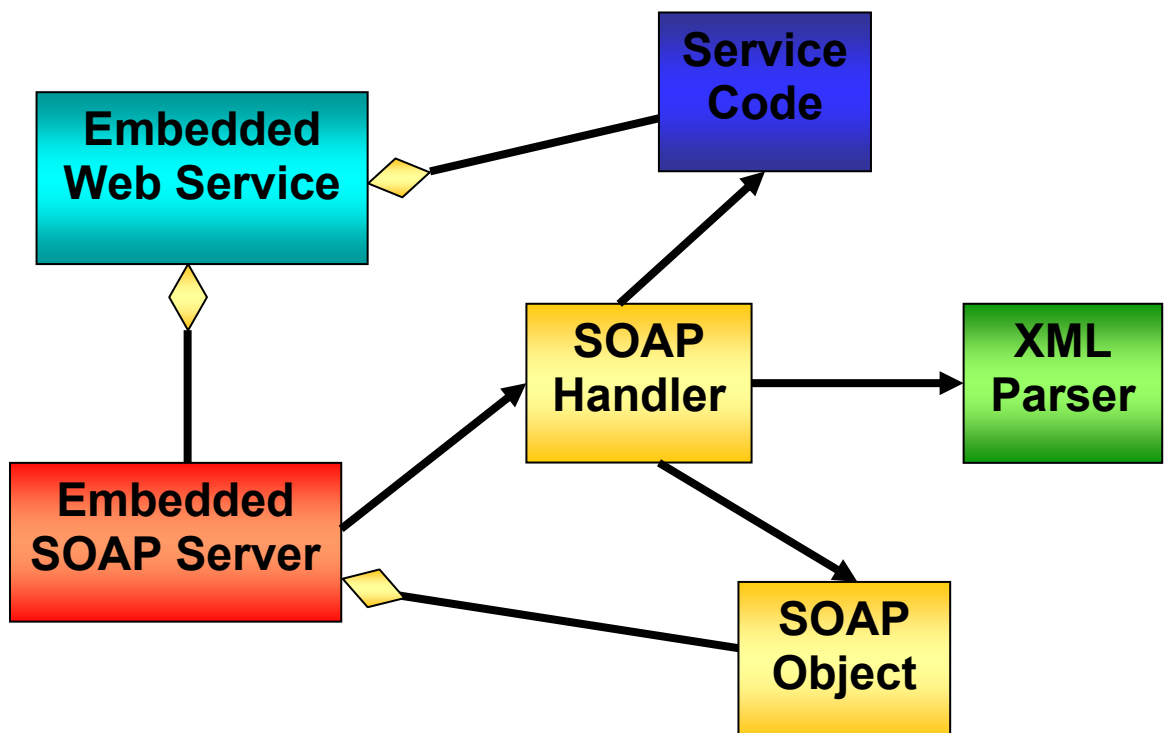
อาจารย์ผู้สืบทอด นิชะโต

ภาควิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์
สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง
ปีการศึกษา 2545

โครงสร้างของเว็บเซอร์วิสบนระบบฝังตัว

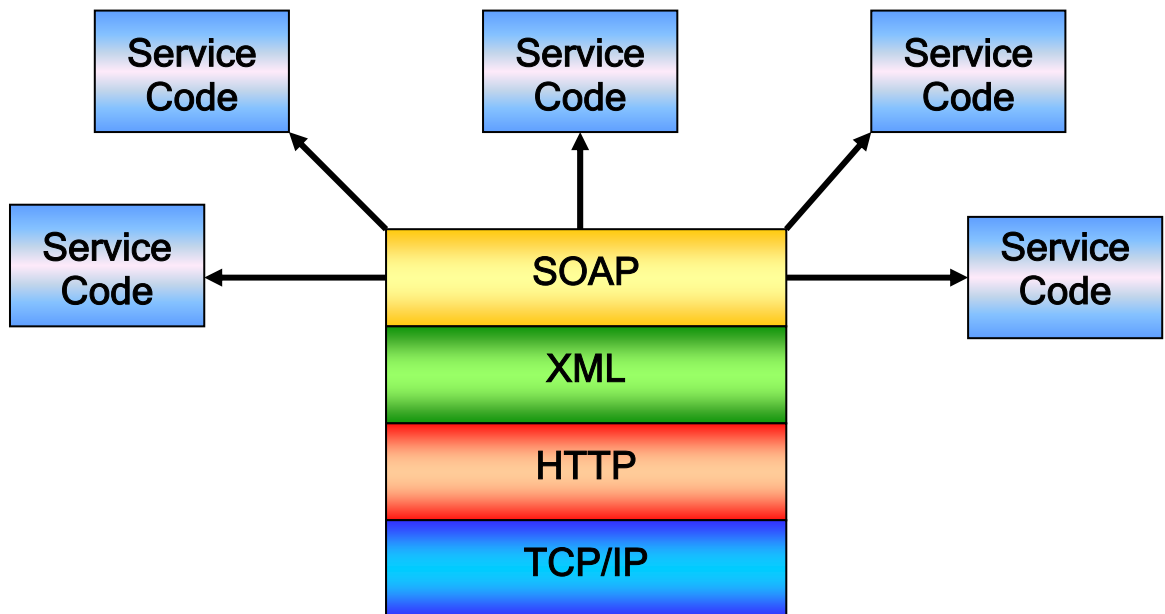
สำหรับโครงสร้างภายในเว็บเซอร์วิสนี้มีส่วนที่สำคัญ คือ

- โซบเซิร์ฟเวอร์บนระบบฝังตัว (Embedded Server) เป็นส่วนที่จัดการเซดเคอร์เอบที่ทีพีและการติดต่อผ่านทางโพรโทคอลทีซีพีไอพี
- โซบแฮนเดิลเลอร์ (SOAP Handler) เป็นส่วนที่ทำหน้าที่จัดการโซบแมสเสจ ซึ่งจะแบ่งเป็น 2 ส่วนคือ ส่วนที่ทำหน้าที่จัดการโซบแมสเสจที่ได้รับมาเมื่อมีการเรียกใช้เซอร์วิสโดยถอดโซบแมสเสจ ให้เป็นรายละเอียดและข้อมูลอินพุตในการเรียกใช้เซอร์วิส และส่วนที่ทำหน้าที่จัดการสร้างโซบแมสเสจเพื่อตอบรับการเรียกใช้เซอร์วิสโดยได้นำเอาผลลัพธ์ที่ได้จากไฟล์เอาท์พุทที่ได้จากการเรียกเซอร์วิสมาเป็นโซบแมสเสจ
- โซบออบเจกต์ (SOAP Object) เป็นส่วนที่ทำหน้าที่เสมือนเป็นส่วนประกอบของโซบแมสเสจซึ่งจะถูกเรียกใช้โดยโซบแฮนเดิลเลอร์และเป็นส่วนหนึ่งของโซบเซิร์ฟเวอร์
- เซอร์วิสโค้ด (Service Code) เป็นตัวแอปพลิเคชันที่อยู่บนตัวอุปกรณ์คอม86 ซึ่งจะถูกเรียกใช้งานโดยส่วนของโซบแฮนเดิลเลอร์
- เอ็กซ์เอ็มแอลพาร์เซอร์ (XML Parser) จะทำหน้าที่ในการแปลเอกสารเอ็กซ์เอ็มแอล โดยที่จะถูกโซบแฮนเดิลเลอร์ นั้นทำการเรียกใช้โซบแฮนเดิลเลอร์ จะทำหน้าที่ในการจัดการโซบ โดยจะถูกเรียกใช้งานโดยโซบเซิร์ฟเวอร์ในระบบฝังตัว



รูปที่ 1 แสดงโครงสร้างภายในระบบเว็บเซอร์วิสบนระบบฝังตัว

โครงสร้างทางสถาปัตยกรรมของเว็บเซอร์วิสบนระบบฝังตัว



รูปที่2 โครงสร้างทางสถาปัตยกรรมของเว็บเซอร์วิสบนระบบฝังตัว

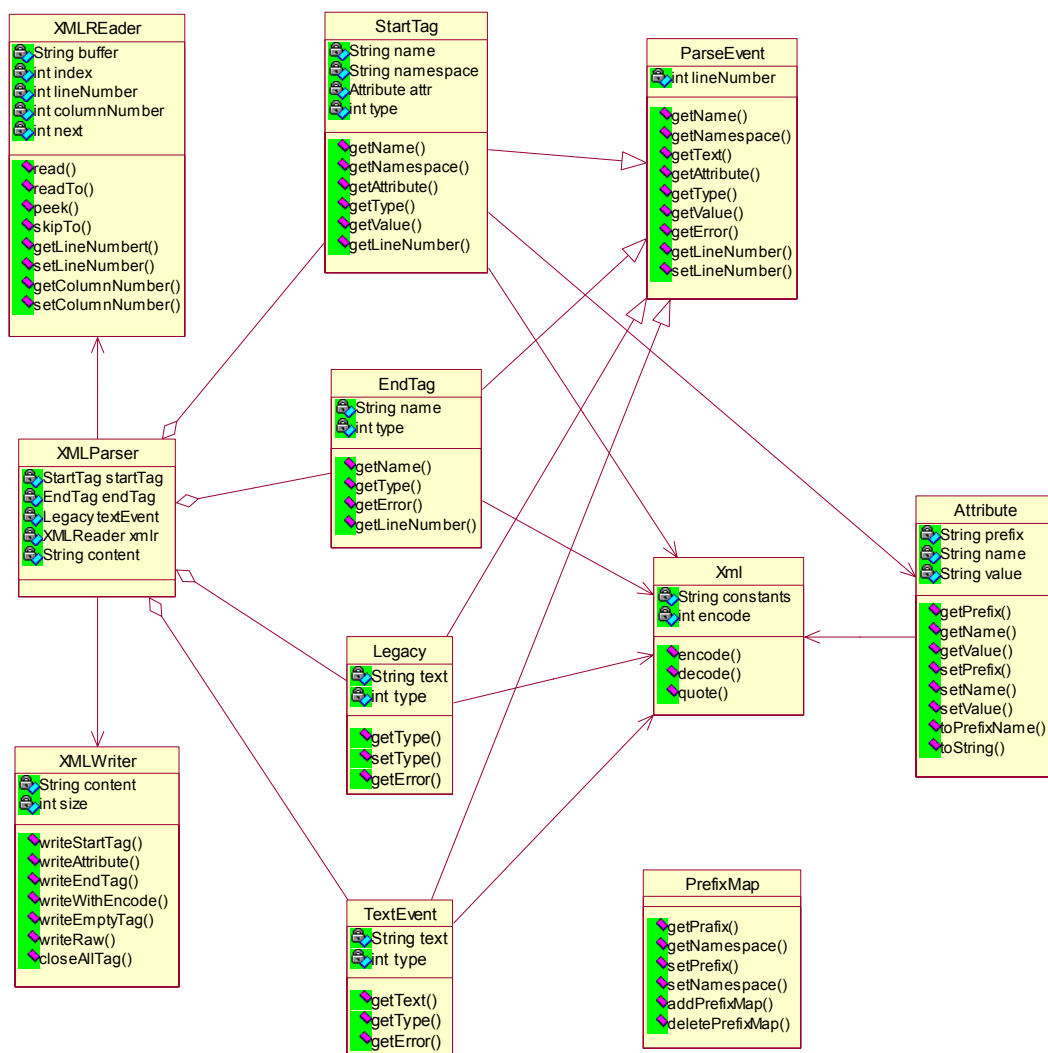
การเขียนโปรแกรมระบบการให้บริการเว็บเซอร์วิสบนระบบฝังตัว

การเขียนโปรแกรมระบบการให้บริการเว็บเซอร์วิสบนระบบฝังตัวนั้น สามารถแบ่งออกเป็น 4 ส่วนที่สำคัญได้แก่

1. ส่วนของเอ็กซ์เอ็มแอลพาร์เซอร์ (XML Parser)
2. ส่วนของโซบพาร์เซอร์ (SOAP Parser)
3. ส่วนของเว็บเซิร์ฟเวอร์ (Web Server)
4. ส่วนของอินเตอร์เฟซของเซอร์วิสโค้ด (Service Code)

1. เอ็กซ์เอ็มแอลพาร์เซอร์

เอ็กซ์เอ็มแอลพาร์เซอร์เป็นส่วนที่ใช้ในการจัดการเอกสารเอ็กซ์เอ็มแอล โดยสามารถอ่านและเขียนข้อมูลในรูปแบบเอ็กซ์เอ็มแอล เนื่องจากโซบแมสเสจใช้โครงสร้างของตามเอกสารเอ็กซ์เอ็มแอลจึงจำเป็นต้องยิ่งที่การสร้างเว็บเซอร์วิสบนระบบฝังตัวจะต้องมีความสามารถในการแปลเอกสารเอ็กซ์เอ็มแอล และเนื่องจากข้อจำกัดในเรื่องของหน่วยความจำจึงจำเป็นต้องยิ่งในการใช้หน่วยความจำให้น้อยที่สุด แต่ต้องคำนึงถึงความสามารถในเรื่องของความเร็วในการให้บริการด้วย



รูปที่ 3 คลาสไดอะแกรมในส่วนของเอ็กซ์เอ็มแอลพาร์เซอร์

1.1 คลาสเอ็กซ์เอ็มแอลรีดเดอร์ (XMLReader Class) เป็นคลาสที่ใช้ในการจัดการกับข้อมูลเอ็กซ์เอ็มแอล โดยจะทำหน้าที่ในส่วนของการอ่านข้อมูลภาษาเอ็กซ์เอ็มแอล

แอททริบิวต์ (Attribute)	
int	next เก็บค่าตัวอักษรที่อยู่ถัดไปของเอกสาร
int	lineNumber เก็บตำแหน่งของบรรทัดของเอกสารที่อ่านอยู่ในปัจจุบัน
int	columnNumber เก็บตำแหน่งของคอลัมน์ของเอกสารที่อ่านอยู่ในปัจจุบัน
int	index เก็บตำแหน่งของตัวอักษรปัจจุบันของเอกสาร
char []	reader เก็บข้อความเอกสารที่ต้องการอ่าน

คอนสตรัคเตอร์ (Constructor)

XMLReader()	สร้างอินสแตนซ์เพื่อทำการกำหนดค่าภายหลังหรือเป็นดีฟอลต์คอนสตรัคเตอร์(Default Constructor)
XMLReader(const char *s)	สร้างอินสแตนซ์โดยทำการกำหนดค่าของเอกสารที่ต้องการอ่านผ่านทางพารามิเตอร์ที่เป็น const char * s

ฟังก์ชัน (Function)	
XMLReader&	Operator=(const XMLReader& value) โอเวอร์โหลดโอเปอเรเตอร์เท่ากับสร้างอินสแตนซ์เอ็กซ์เอ็มแอลรีดเดอร์ที่มีค่าของแอตทริบิวต์ภายในเท่ากับอินสแตนซ์ที่กำหนดในส่วนของพารามิเตอร์ const XMLReader& value
int	eof() ตรวจสอบว่าเป็นตำแหน่งสุดท้ายของเอกสารหรือไม่ ถ้าเป็นจะส่งค่ากลับเป็น int ที่มีค่าเป็น 1 และถ้ายังไม่เป็นจะมีค่าเป็น 0
int	getLineNumber() ส่งค่ากลับเป็น int ที่บอกถึงตำแหน่งของบรรทัดของเอกสารที่กำลังถูกอ่านอยู่ในปัจจุบัน
int	getColumnNumber() ส่งค่ากลับเป็น int ที่บอกถึงตำแหน่งของคอลัมน์ของเอกสารที่กำลังถูกอ่านอยู่ในปัจจุบัน
const char*	getContent() ส่งข้อความที่อยู่ในคลาสกลับเป็น const char*
char*	getContentNotRead() ส่งค่ากลับเป็น char* ที่เก็บข้อความที่อยู่ในคลาสส่วนที่ยังไม่ได้ทำการอ่าน
int	getIndex() ส่งค่ากลับเป็น int ที่บอกถึงตำแหน่งของตัวอักษรปัจจุบันของเอกสาร
void	skipWhitespace() ทำการอ่านข้ามเอกสารจนอักษรที่อ่านไม่เป็นอักษรที่เป็นช่องว่าง
int	read(char* buf, int off, int len) ทำการอ่านเอกสารเอ็กซ์เอ็มแอลแล้วทำการเขียนลงในพารามิเตอร์ที่กำหนดโดย char* buf ตำแหน่งเริ่มต้นที่กำหนดโดย int off และตำแหน่งสุดท้ายกำหนดโดย int len ส่วนการส่งค่ากลับของฟังก์ชันจะเป็นค่าที่บอกถึงสถานะภาพของการเรียกใช้ฟังก์ชัน ถ้าสามารถทำงานได้อย่างถูกต้องจะส่งค่ากลับเป็น 1 แต่ถ้าไม่สามารถทำงานได้ตามปกติจะส่งค่ากลับเป็น 0
char*	readN(int count) ทำการอ่านเอกสารจากตำแหน่งปัจจุบันไปตามจำนวน int count ที่กำหนดในส่วนของพารามิเตอร์
char*	readIdentifier() ทำการอ่านเอกสารเฉพาะส่วนที่เป็นตัวอักษรและตัวเลขแล้วทำการส่งค่ากลับเป็นพอยเตอร์ของตัวอักษร
char*	readTo(const char* stopChars) ทำการอ่านเอกสารจนถึงตำแหน่งที่มี

	ตัวอักษรเหมือนกับตัวอักษรภายในข้อความที่กำหนดโดยพารามิเตอร์ const char * stopChars แล้วส่งค่ากลับเป็นข้อความที่ได้อ่านแล้วในรูป ของ char*
char*	readTo (const char stopChar) ทำการอ่านเอกสารจนถึงตำแหน่งที่มี ตัวอักษรที่เหมือนกับตัวอักษรที่กำหนดโดยพารามิเตอร์ const char stopChar แล้วส่งค่ากลับข้อความที่ได้อ่านแล้วในรูปของ char*
int	skipTo (const char* stopChars) ทำการอ่านข้ามเอกสารจนถึงตำแหน่งที่ มีตัวอักษรเหมือนกับข้อความที่กำหนดในส่วนของพารามิเตอร์ const char* stopChars แล้วส่งค่ากลับเป็นค่าตัวอักษรตัวถัดไปในรูปของ int
char*	readWhile (const char* acceptChars) ทำการอ่านเอกสารในขณะที่ ตัวอักษรที่อ่านมีค่าเหมือนกับตัวอักษรที่อยู่ภายในข้อความที่กำหนดใน ส่วนของพารามิเตอร์ const char* acceptChars แล้วส่งค่ากลับเป็น ข้อความที่ได้อ่านแล้วในรูปของ char*
int	peek () ทำการส่งค่ากลับเป็นค่าของตัวอักษรถัดไปในรูปของ int
void	setContent(const char* string) ทำกำหนดเอกสารที่ต้องการอ่านให้กับ อินสแตนซ์ซึ่งกำหนดโดยพารามิเตอร์ const char* string
int	read () ทำการอ่านตัวอักษรถัดไปแล้วส่งค่ากลับในรูปของ int

1.2 คลาสเอ็กซ์เอ็มแอลพาร์เซอร์ (XMLParser Class) เป็นคลาสที่ใช้ในการจัดการกับข้อมูลเอ็กซ์เอ็มแอล โดยจะทำหน้าที่ในส่วนของการแปลงข้อมูลภาษาเอ็กซ์เอ็มแอล ซึ่งจะเรียกใช้คลาสเอ็กซ์เอ็มแอลไรท์เทอร์ และ เอ็กซ์เอ็มแอลรีดเดอร์

แอททริบิวต์ (Attribute)	
int	size เก็บค่าขนาดของเอกสารภายในอินสแตนซ์
ParseEvent*	res เก็บค่าพอยน์เตอร์ไปยังคลาสพาสอีเวนต์
XMLReader	reader ใช้ในการอ่านเอกสาร
Str	qNames[] เก็บค่าของควอนทีไฟลีนัม
char []	reader เก็บข้อความเอกสารที่ต้องการอ่าน
StartTag	startTag เก็บค่าอินสแตนซ์ของคลาสสตาร์ทแท็กที่ได้จากการอ่านเอกสาร
EndTag	endTag เก็บค่าอินสแตนซ์ของคลาสเอ็นแท็กที่ได้จากการอ่านเอกสาร
EndDocument	endDoc เก็บค่าอินสแตนซ์ของคลาสเอ็นด็อกูเมนต์ที่ได้จากการอ่านเอกสาร
LegacyEvent	legacy เก็บค่าอินสแตนซ์ของคลาสเลกาซีอีเวนต์ที่ได้จากการอ่านเอกสาร
TextEvent	TextEvent เก็บค่าอินสแตนซ์ของคลาสเท็กส์อีเวนต์ที่ได้จากการอ่านเอกสาร

PrefixMap	prefixMap เก็บค่าอินสแตนซ์ของคลาสพรีฟิกแมปที่ได้จากการอ่านเอกสาร
-----------	--

คอนสตรัคเตอร์ (Constructor)	
Parser()	สร้างอินสแตนซ์เพื่อทำการกำหนดค่าภายหลังหรือเป็นดีฟอลท์คอนสตรัคเตอร์(Default Constructor)
Parser (const char* r)	สร้างอินสแตนซ์โดยทำการกำหนดค่าของเอกสารที่ต้องการพาสภายในอินสแตนซ์ของคลาสด้วยพารามิเตอร์ const char* r

ฟังก์ชัน (Function)	
void	setProcessNamespaces (int processNs) ใช้ในการกำหนดค่าของแอตทริบิวต์ processNamespaces ภายในคลาสที่กำหนดโดยพารามิเตอร์ int processNs
Void	setXMLReader (const char* r) ใช้ในการกำหนดค่าของเอกสารภายในของแอตทริบิวต์ XMLReader reader ภายในคลาสที่กำหนดโดยพารามิเตอร์ const char* r
char*	readText () ใช้ในการอ่านข้อความภายในจนกว่าจะเจอส่วนของ EndTag หรือส่วนของ EndDocument และส่งค่ากลับเป็นข้อความที่อ่านในรูปแบบของ char*
const char	parseCharacterEntity () ใช้ในการจัดการพาสตัวอักษรที่เป็นเอ็นดีดีและส่งค่ากลับเป็น const char
LegacyEvent	parseComment () ใช้ในการจัดการพาสส่วนคอมเมนต์ และส่งค่ากลับเป็นอินสแตนซ์ของคลาส LegacyEvent
LegacyEvent	parseDoctype () ใช้ในการจัดการพาสส่วนของคอกูเมนต์ไทป์ และส่งค่ากลับเป็นอินสแตนซ์ของคลาส LegacyEvent
TextEvent	parseCDATA () ใช้ในการจัดการพาสส่วนของซีดาต้า และส่งค่ากลับเป็นอินสแตนซ์ของคลาส TextEvent
EndTag	parseEndTag () ใช้ในการพาสส่วนของเอ็นแท็ก และส่งค่ากลับเป็นอินสแตนซ์ของคลาส EndTag
LegacyEvent	parsePI () ใช้ในการพาสส่วนของโปรเซสซิ่งอินสตรักชัน และส่งค่ากลับเป็นอินสแตนซ์ของคลาส LegacyEvent
StartTag	parseStartTag () ใช้ในการพาสส่วนของสตาร์ทแท็ก และส่งค่ากลับเป็นอินสแตนซ์ของคลาส StartTag
TextEvent	parseText () ใช้ในการพาสส่วนของเท็กซ์อีเวนต์ และส่งค่ากลับเป็น

	อินสแตนซ์ของคลาส TextEvent
void	parseSpecial () ใช้ในการเลือกฟังก์ชันที่ใช้ในการพาสเอกสารที่มีรูปแบบพิเศษที่แตกต่างกันในกรณีเริ่มต้นด้วยเครื่องหมาย <
ParseEvent*	peek () ใช้ในการเรียกดูส่วนที่อยู่ถัดไปของการพาส และส่งค่ากลับเป็น ParseEvent*
int	isRead () ใช้ในการตรวจสอบว่าสามารถที่จะอ่านต่อไปได้หรือไม่ และส่งค่ากลับเป็น int โดยถ้าเป็น 1 แสดงว่าสามารถอ่านต่อไปได้ และถ้าเป็น 0 ไม่สามารถอ่านต่อไปได้
ParseEvent*	read () ใช้ในการอ่านเอกสารเอ็กซ์เอ็มแอล และส่งค่ากลับเป็น ParseEvent*
int	getLineNumber () const ใช้ในการเรียกดูค่าของบรรทัดของเอกสารที่กำลังอ่านอยู่ และส่งค่ากลับเป็น int

1.3 คลาสเอ็กซ์เอ็มแอลไรท์เทอร์ (XMLWriter Class) เป็นคลาสที่ใช้ในการจัดการกับข้อมูลเอ็กซ์เอ็มแอล โดยจะทำหน้าที่ในส่วนของการเขียนข้อมูลภาษาเอ็กซ์เอ็มแอล

แอตทริบิวต์ (Attribute)	
int	countns เก็บจำนวนนับของเนมสเปซของเอกสาร
int	Pending เก็บค่าที่บอกถึงค่าที่บอกถึงสถานะของแท็กว่าได้ถูกเปิดอยู่หรือไม่ โดยถ้ามีค่าเป็น 1 หมายถึงได้ทำการเขียนเครื่องหมาย < ไว้แล้วยังไม่ได้ทำการปิด และถ้ามีค่าเป็น 0 มีความหมายตรงกันข้าม
char	writer[] เก็บค่าของเอกสารเอ็กซ์เอ็มแอลที่สร้างขึ้น
Str	nametag[] เก็บค่าของลำดับของแท็กเนมที่ถูกเขียนลงในเอกสารเอ็กซ์เอ็มแอลที่สร้างขึ้นเพื่อตรวจสอบความถูกต้องของเอกสาร
Str	error เก็บค่าความผิดพลาดที่เกิดขึ้นจากการสร้างเอกสารเอ็กซ์เอ็มแอล

คอนสตรัคเตอร์ (Constructor)	
XMLWriter()	สร้างอินสแตนซ์เพื่อทำการกำหนดค่าภายหลังหรือเป็นดีฟอลท์คอนสตรัคเตอร์(Default Constructor)
XMLWriter(const char *w)	สร้างอินสแตนซ์โดยทำการกำหนดค่าของเอกสารเอ็กซ์เอ็มแอลที่ถูกเขียนไว้แล้วและต้องการทำการเขียนต่อซึ่งกำหนดโดยพารามิเตอร์ const

	char* w
--	---------

ฟังก์ชัน (Function)	
const char*	getWriter () const เป็นคอนสแตนต์ฟังก์ชันที่ใช้ในการเรียกดูเอกสาร เอ็กซ์เอ็มแอลที่สร้างขึ้นมา โดยการส่งค่ากลับเป็น const char*
void	checkPending () ใช้ในการตรวจสอบค่า pending ถ้าค่าเท่ากับ 1 จะทำ การเขียนเครื่องหมายมากกว่า (>) และ ‘\n’ ลงในเอกสารเอ็กซ์เอ็มแอล
void	write (char c) ใช้ในการเขียนตัวอักษรลงในเอกสารเอ็กซ์เอ็มแอลโดย การเปลี่ยนตัวอักษรทั้ง 3 ตัวที่เป็นเอ็นดีรีเฟอเรนซ์ให้อยู่ในรูปแบบ ที่ถูกต้องซึ่งได้แก่ <, >, &
void	write (char* buf, int start, int len) ใช้ในการเขียนข้อความลงในเอกสาร เอ็กซ์เอ็มแอล โดยการกำหนดพารามิเตอร์เป็น char* buf ซึ่งเป็นตัวแปร ที่ใช้ในการเก็บข้อความที่ต้องการเขียนลงในเอกสาร และ int start ที่ บอกถึงตำแหน่งเริ่มต้นของข้อความ และ int len ที่บอกถึงความยาวของ ข้อความที่ต้องการเขียน
void	writeAttribute (const char* attr) ใช้ในการเขียนแอตทริบิวต์ในส่วน ของสตาร์ทแท็กในเอกสารเอ็กซ์เอ็มแอล โดยการกำหนดพารามิเตอร์เป็น const char* attr ที่บอกถึงข้อความแอตทริบิวต์ซึ่งได้จัดให้อยู่ในรูปแบบ ที่ถูกต้อง
void	writeAttribute (const char* pf, const char* n, const char* v) ใช้ในการ เขียนแอตทริบิวต์ในส่วนของสตาร์ทแท็กในเอกสารเอ็กซ์เอ็มแอล โดย การกำหนดพารามิเตอร์เป็นคอนสแตนต์ของพอยน์เตอร์ทูลอาร์ 3 ตัวคือ pf กำหนดค่าพรีฟิกแอตทริบิวต์, n กำหนดค่าของชื่อแอตทริบิวต์ และ v กำหนดค่าของแอตทริบิวต์
void	writeAttribute (const char* n, const char* v) ใช้ในการเขียนแอตทริบิวต์ ในส่วนของสตาร์ทแท็กในเอกสารเอ็กซ์เอ็มแอล โดยการกำหนด พารามิเตอร์เป็นคอนสแตนต์ของพอยน์เตอร์ทูลอาร์ 2 ตัวคือ n กำหนดค่า ของชื่อแอตทริบิวต์ และ v กำหนดค่าของแอตทริบิวต์
void	writeStartTag (const char* pf, const char* n) ใช้ในการเขียนสตาร์ท แท็ก โดยการกำหนดพารามิเตอร์ const char* pf ที่บอกถึงค่าพรีฟิกของ สตาร์ทแท็ก และ const char* n ที่บอกถึงชื่อของสตาร์ทแท็ก
void	writeStartTag (const char* t) ใช้ในการเขียนสตาร์ทแท็ก โดยการ กำหนดพารามิเตอร์ const char* t ที่บอกถึงข้อความแท็กหรือชื่อแท็ก
void	writeEmptyTag (const char* pf, const char* n, const char* attr) ใช้ใน

	การเขียนเอ็มดีแท็ก โดยการกำหนดพารามิเตอร์ <code>const char* pf</code> ที่บอกถึงค่าพรีฟิกของสตาร์ทแท็ก, <code>const char* n</code> ที่บอกถึงชื่อของแท็ก และ <code>const char* attr</code> ที่บอกถึงข้อความแอตทริบิวต์ที่อยู่ในรูปแบบที่ต้องการ
void	<code>writeEmptyTag (const char* tag, const char* attr)</code> ใช้ในการเขียนเอ็มดีแท็ก โดยการกำหนดพารามิเตอร์ <code>const char* tag</code> ที่บอกถึงข้อความแท็ก และ <code>const char* attr</code> ที่บอกถึงข้อความแอตทริบิวต์ที่อยู่ในรูปแบบที่ต้องการ
void	<code>writeEndTag (const char* tag)</code> ใช้ในการเขียนเอ็นแท็ก โดยการกำหนดพารามิเตอร์ <code>const char* tag</code> ที่บอกถึงข้อความแท็ก
void	<code>writeEndTag ()</code> ใช้ในการเขียนเอ็นแท็ก โดยชื่อของแท็กนั้นนำมาจากค่าของอาร์เรย์ของชื่อแท็กซึ่งเป็นแอตทริบิวต์ภายในคลาส
void	<code>writeTag (const char* tag, const char* attr, const char* value)</code> ใช้ในการเขียนแท็กทั้งส่วนของแท็กเปิด แท็กปิด และค่าภายในแท็ก โดยการกำหนดพารามิเตอร์ <code>const char* tag</code> ที่บอกถึงข้อความแท็กที่ต้องการเขียน, <code>const char* attr</code> ที่บอกถึงข้อความแอตทริบิวต์ภายในสตาร์ทแท็ก, <code>const char* value</code> ที่บอกถึงข้อความของค่าภายในแท็ก
void	<code>closeAll ()</code> ใช้ในการเขียนแท็กปิดที่ค้างอยู่ทั้งหมดภายในอาร์เรย์ของชื่อแท็กที่เหลืออยู่
void	<code>writeLegacy (int type,const char* content)</code> ใช้ในการเขียนส่วนเลกาซีที่มีอยู่ 3 รูปแบบคือ ส่วนของคอมเมนต์ ส่วนของโปรเซสเซอร์อินสตรักชัน และส่วนของการกำหนดโครงสร้างของเอกสารเอ็กซ์เอ็มแอล ที่กำหนดโดยพารามิเตอร์ <code>int type</code> ที่บอกถึงรูปแบบที่ต้องการเขียน, <code>const char* content</code> ที่บอกถึงข้อความภายในที่ต้องการเขียน
void	<code>writeRaw (const char* s)</code> ใช้ในการเขียนข้อมูลทั่วไปลงไปเอกสารโดยตรงไม่ต้องทำการเปลี่ยนแปลงกำหนดโดยพารามิเตอร์ <code>const char* s</code>

1.4 คลาสสตาร์ทแท็ก (StartTag Class) เป็นคลาสที่ใช้ในการจัดการกับแท็กข้อมูลของเอ็กซ์เอ็มแอล โดยทำหน้าที่ในส่วนofdตัวสตาร์ทแท็กเอ็กซ์เอ็มแอล เช่น ตรวจสอบความถูกต้องของส่วนสตาร์ทแท็กตามรูปแบบของภาษาเอ็กซ์เอ็มแอล

แอตทริบิวต์ (Attribute)	
Str	name เก็บค่าของชื่อแท็ก
Str	name_space เก็บค่าของเนมสเปซของแท็ก
AttributeHandler	attrHandler เก็บอินสแตนซ์ของคลาสที่ใช้ในการจัดการค่าของแอตทริบิวต์

int	degenerated เก็บค่าที่บอกถึงเอ็มดีแท็ก โดยถ้าเป็น 1 เป็นเอ็มดีแท็ก ถ้าเป็น 0 เป็นสตาร์ทแท็ก
Str	error เก็บข้อความที่บ่งบอกความผิดพลาดที่เกิดขึ้นกับอินสแตนซ์ของคลาสที่สร้างขึ้น

คอนสตรัคเตอร์ (Constructor)	
StartTag ()	สร้างอินสแตนซ์เพื่อทำการกำหนดค่าภายหลังหรือเป็นคิฟอลท์คอนสตรัคเตอร์(Default Constructor)
StartTag (const char* ns, const char * n, int degen)	สร้างอินสแตนซ์โดยทำการกำหนดพารามิเตอร์ const char* ns ที่บอกถึงค่าของเนมสเปซ, const char* n ที่บอกถึงชื่อของแท็ก และ int degen ที่บอกถึงบอกความเป็นเอ็มดีแท็ก
StartTag (XMLReader reader)	สร้างอินสแตนซ์โดยทำการกำหนดค่าอินสแตนซ์ของคลาสที่กำหนดจากพารามิเตอร์ XMLReader reader ซึ่งจะทำการอ่านเอกสารโดยเรียกใช้อินสแตนซ์ของคลาสในการนำเอารายละเอียดออกมาเพื่อทำการกำหนดรายละเอียดต่างของสตาร์ทแท็ก

ฟังก์ชัน (Function)	
StartTag&	operator= (const StartTag& value) โอเวอร์โหลดโอเปอร์เรเตอร์ที่ใช้ในการก๊อปปี้ค่าของอินสแตนซ์ของคลาสที่กำหนดโดยพารามิเตอร์ const StartTag& value และส่งค่ากลับเป็น StartTag&
const AttributeHandler	getAttributeHandler() const เป็นคอนสแตนซ์ฟังก์ชันใช้ในการเรียกดูค่าอินสแตนซ์ของคลาส AttributeHandler ที่เป็นแอตทริบิวต์ภายในคลาส และทำการส่งค่ากลับเป็น const AttributeHandler
char*	getAttribute () const เป็นคอนสแตนซ์ฟังก์ชันใช้ในการเรียกดูข้อความแอตทริบิวต์ที่มีอยู่ในอินสแตนซ์ของคลาส AttributeHandler และทำการส่งค่ากลับเป็น char*
int	getCountAttr () const เป็นคอนสแตนซ์ฟังก์ชันใช้ในการเรียกดูจำนวนนับของแอตทริบิวต์ที่อยู่ภายในสตาร์ทแท็ก และทำการส่งค่ากลับเป็น int
const char*	getAttribute (int index) const เป็นคอนสแตนซ์ฟังก์ชันใช้ในการเรียกดูข้อความแอตทริบิวต์ภายในที่อยู่ในสตาร์ทแท็กเฉพาะลำดับที่กำหนดโดยพารามิเตอร์ int index และทำการส่งค่าเป็น const char*
int	getDegenerated () const เป็นคอนสแตนซ์ฟังก์ชันใช้ในการเรียกดูค่า degenerated ซึ่งเป็นแอตทริบิวต์ภายในคลาส และส่งค่ากลับเป็น int
const char*	getName () const เป็นคอนสแตนซ์ฟังก์ชันที่ใช้ในการเรียกดูชื่อของ

	แท็ก และส่งค่ากลับเป็น const char*
const char*	getNamespace () const เป็นคอนสแตนต์ฟังก์ชันที่ใช้ในการเรียกดูเนมสเปซของแท็ก และส่งค่ากลับเป็น const char*
int	getType () const เป็นคอนสแตนต์ฟังก์ชันที่ใช้ในการส่งค่ากลับเป็นค่าที่บอกถึงชนิดของอีเวนต์ที่เกิดจากการพาส (ParseEvent)
char*	getValue (const char* attrName) const เป็นคอนสแตนต์ฟังก์ชันที่ใช้ในการเรียกดูค่าของข้อความแอตทริบิวต์ที่กำหนดโดยพารามิเตอร์ const attrName ซึ่งอยู่ภายในอินสแตนซ์ของคลาส AttributeHandler และส่งค่ากลับเป็น char*
const char*	getText () const เป็นคอนสแตนต์ฟังก์ชันที่จะส่งค่ากลับเป็น NULL เพราะเป็นสตาร์ทแท็กซึ่งจะไม่มีข้อความภายใน
int	endCheck (ParseEvent* start) ใช้ในการส่งค่ากลับเป็น 0
void	setError (const char* e) ใช้ในการกำหนดค่าของข้อความผิดพลาด ซึ่งกำหนดโดยพารามิเตอร์ const char* e ที่บอกถึงข้อความบอกความผิดพลาดที่ต้องการกำหนดให้กับแอตทริบิวต์ Str error ของคลาส
const char*	getError () const เป็นคอนสแตนต์ฟังก์ชันที่ใช้ในการเรียกดูข้อความบอกความผิดพลาดของคลาสที่มีอยู่ปัจจุบัน และส่งค่ากลับเป็น const char*
char*	toString () ใช้ในการเรียกดูข้อความของสตาร์ทแท็กในรูปแบบของเอกสารเอ็กซ์เอ็มแอล และส่งค่ากลับเป็น char*

1.5 คลาสเอนแท็ก (EndTag Class) เป็นคลาสที่ใช้ในการจัดการกับแท็กข้อมูลของเอ็กซ์เอ็มแอล โดยทำหน้าที่ในส่วนของตัวเองเอนแท็กเอ็กซ์เอ็มแอล เช่น เช็คความถูกต้องของตัวเองเอนแท็กเอ็กซ์เอ็มแอล

แอตทริบิวต์ (Attribute)	
Str	endTag เก็บชื่อของเอนแท็ก
Str	error เก็บข้อความที่บ่งบอกความผิดพลาดที่เกิดขึ้นกับอินสแตนซ์ของคลาสที่สร้างขึ้น

คอนสตรัคเตอร์ (Constructor)	
EndTag ()	สร้างอินสแตนซ์เพื่อทำการกำหนดค่าภายหลังหรือเป็นดีฟอลท์คอนสตรัคเตอร์(Default Constructor)
EndTag(const char *s)	สร้างอินสแตนซ์โดยทำการกำหนดค่าของชื่อของเอนแท็กจากพารามิเตอร์ const char* s

ฟังก์ชัน (Function)	
EndTag&	Operator=(const EndTag& value) โอเวอร์โหลดโอเปอเรเตอร์เท่ากับ สร้างอินสแตนซ์ของคลาสเอ็นแท็กให้มีค่าของแอตทริบิวต์ภายใน เท่ากับอินสแตนซ์ที่กำหนดที่กำหนดโดยพารามิเตอร์ const EndTag& value
const char*	getEndTag () const เป็นคอนสแตนต์ฟังก์ชันที่ใช้ในการเรียกดูชื่อความ ชื่อของคลาสเอ็นแท็กในรูปของ const char*
char*	getAttribute () const เป็นคอนสแตนต์ฟังก์ชันที่จะส่งค่ากลับเป็น NULL
int	getCountAttr () const เป็นคอนสแตนต์ฟังก์ชันที่จะส่งค่ากลับเป็น 0
const char*	getAttribute (int index) const เป็นคอนสแตนต์ฟังก์ชันที่จะส่งค่ากลับ เป็น NULL
char*	getValue (const char* attrName) const เป็นคอนสแตนต์ฟังก์ชันที่ใช้ใน การส่งค่ากลับเป็น NULL
const char*	getText () const เป็นคอนสแตนต์ฟังก์ชันที่จะส่งค่ากลับเป็น NULL เพราะเป็นเอ็นแท็กซึ่งจะไม่มีข้อความภายใน
int	endCheck (ParseEvent* start) ใช้ในการส่งค่ากลับเป็น 1
void	setError (const char* e) ใช้ในการกำหนดค่าของข้อความผิดพลาด ซึ่ง กำหนดโดยพารามิเตอร์ const char* e ที่บอกถึงข้อความบอกความ ผิดพลาดที่ต้องการกำหนดให้กับแอตทริบิวต์ Str error ของคลาส
const char*	getError () const เป็นคอนสแตนต์ฟังก์ชันที่ใช้ในเรียกดูข้อความบอก ความผิดพลาดของคลาสที่มีอยู่ปัจจุบัน และส่งค่ากลับเป็น const char*
char*	toString () ใช้ในการเรียกดูข้อความของเอ็นแท็กในรูปแบบของเอกสาร เอ็กซ์เอ็มแอล และส่งค่ากลับเป็น char*

1.6 คลาสเลกกาซี (Legacy Class) เป็นคลาสที่ใช้ในการจัดการกับข้อมูลของเอ็กซ์เอ็มแอล โดยทำหน้าที่ ในส่วนของตัวข้อมูลของเอ็กซ์เอ็มแอลที่มีอยู่แล้ว

แอตทริบิวต์ (Attribute)	
int	type เก็บชนิดของรูปแบบเลขชี้กำลังของอินสแตนซ์ของคลาส
Str	text เก็บข้อความภายในของคลาสที่มีอยู่
Str	eror เก็บข้อความที่บ่งบอกความผิดพลาดที่เกิดขึ้นกับอินสแตนซ์ของคลาสที่สร้างขึ้น

คอนสตรัคเตอร์ (Constructor)	
Legacy()	สร้างอินสแตนซ์เพื่อทำการกำหนดค่าภายหลังหรือเป็นดีฟอลต์คอนสตรัคเตอร์(Default Constructor)
Legacy(const char *s)	สร้างอินสแตนซ์โดยทำการกำหนดค่าภายในของอินสแตนซ์ของคลาสเลขาซึ่งจากพารามิเตอร์ const char* s

ฟังก์ชัน (Function)	
Legacy&	Operator=(const Legacy& value) โอเวอร์โหลดโอเปอเรเตอร์เท่ากับสร้างอินสแตนซ์เอ็กซ์เอ็มแอลรีดเดอร์ที่มีค่าของแอตทริบิวต์ภายในเท่ากับอินสแตนซ์ที่กำหนดโดยพารามิเตอร์ const Legacy& value
char*	getAttribute () const เป็นคอนสแตนซ์ฟังก์ชันที่จะส่งค่ากลับเป็น NULL
int	getCountAttr () const เป็นคอนสแตนซ์ฟังก์ชันที่จะส่งค่ากลับเป็น 0
const char*	getAttribute (int index) const เป็นคอนสแตนซ์ฟังก์ชันที่จะส่งค่ากลับเป็น NULL
char*	getValue (const char* attrName) const เป็นคอนสแตนซ์ฟังก์ชันที่ใช้ในการส่งค่ากลับเป็น NULL
const char*	getText () const เป็นคอนสแตนซ์ฟังก์ชันใช้ในการเรียกดูค่าภายในของคลาสเลขาซึ่งในรูปแบบของ const char*
int	endCheck (ParseEvent* start) ใช้ในการส่งค่ากลับเป็น 0
void	setError (const char* e) ใช้ในการกำหนดค่าของข้อความผิดพลาด ซึ่งกำหนดโดยพารามิเตอร์ const char* e ที่บอกถึงข้อความบอกความผิดพลาดที่ต้องการกำหนดให้กับแอตทริบิวต์ Str error ของคลาส
const char*	getError () const เป็นคอนสแตนซ์ฟังก์ชันที่ใช้ในเรียกดูข้อความบอกความผิดพลาดของคลาสที่มีอยู่ปัจจุบัน และส่งค่ากลับเป็น const char*
char*	toString () ใช้ในการเรียกดูข้อความของเลขาซึ่งในรูปแบบของเอกสารเอ็กซ์เอ็มแอล และส่งค่ากลับเป็น char*

1.7 คลาสเท็กซ์อีเวนต์ (TextEvent Class) เป็นคลาสที่ใช้ในการจัดการกับข้อมูลของเอ็กซ์เอ็มแอล โดยทำหน้าที่ในส่วนอีเวนต์ของเท็กซ์เอ็กซ์เอ็มแอลที่ส่งเข้ามา

แอตทริบิวต์ (Attribute)	
Str	text เก็บข้อความภายในของคลาสที่มีอยู่
Str	error เก็บข้อความที่บ่งบอกความผิดพลาดที่เกิดขึ้นกับอินสแตนซ์ของคลาสที่สร้างขึ้น

คอนสตรัคเตอร์ (Constructor)	
TextEvent()	สร้างอินสแตนซ์เพื่อทำการกำหนดค่าภายหลังหรือเป็นดีฟอลท์คอนสตรัคเตอร์(Default Constructor)
TextEvent(const char *s)	สร้างอินสแตนซ์โดยทำการกำหนดค่าภายในของอินสแตนซ์ของคลาสได้จากพารามิเตอร์ const char* s

ฟังก์ชัน (Function)	
TextEvent&	Operator=(const TextEvent& value) โอเวอร์โหลดโอเปอเรเตอร์เท่ากับสร้างอินสแตนซ์เอ็กซ์เอ็มแอลรีดเดอร์ที่มีค่าของแอตทริบิวต์ภายในเท่ากับอินสแตนซ์ที่กำหนด
char*	getAttribute () const เป็นคอนสแตนซ์ฟังก์ชันที่จะส่งค่ากลับเป็น NULL
int	getCountAttr () const เป็นคอนสแตนซ์ฟังก์ชันที่จะส่งค่ากลับเป็น 0
const char*	getAttribute (int index) const เป็นคอนสแตนซ์ฟังก์ชันที่จะส่งค่ากลับเป็น NULL
char*	getValue (const char* attrName) const เป็นคอนสแตนซ์ฟังก์ชันที่ใช้ในการส่งค่ากลับเป็น NULL
const char*	getText () const เป็นคอนสแตนซ์ฟังก์ชันใช้ในการเรียกดูค่าภายในของคลาสเท็กซ์โอเวนต์ในรูปแบบของ const char*
int	endCheck (ParseEvent* start) ใช้ในการส่งค่ากลับเป็น 0
void	setError (const char* e) ใช้ในการกำหนดค่าของข้อความผิดพลาด ซึ่งกำหนดโดยพารามิเตอร์ const char* e ที่บอกถึงข้อความบอกความผิดพลาดที่ต้องการกำหนดให้กับแอตทริบิวต์ Str error ของคลาส
const char*	getError () const เป็นคอนสแตนซ์ฟังก์ชันที่ใช้ในเรียกดูข้อความบอกความผิดพลาดของคลาสที่มีอยู่ปัจจุบัน และส่งค่ากลับเป็น const char*
char*	toString () ใช้ในการเรียกดูข้อความของเท็กซ์โอเวนต์ในรูปแบบของเอกสารเอ็กซ์เอ็มแอล และส่งค่ากลับเป็น char*

1.8 คลาสพาสอีเวนต์ (ParseEvent Class) เป็นคลาสที่ใช้ในการจัดการกับข้อมูลของเอ็กซ์เอ็มแอล โดยทำหน้าที่ในแปลข้อมูลส่วนอีเวนต์ของเท็กซ์เอ็กซ์เอ็มแอลที่ส่งเข้ามา

แอตทริบิวต์ (Attribute)

int	next เก็บค่าตัวอักษรที่อยู่ถัดไปของเอกสาร
int	lineNumber เก็บตำแหน่งของบรรทัดของเอกสารที่อ่านอยู่ในปัจจุบัน
int	columnNumber เก็บตำแหน่งของคอลัมน์ของเอกสารที่อ่านอยู่ในปัจจุบัน
int	index เก็บตำแหน่งของตัวอักษรปัจจุบันของเอกสาร
char []	reader เก็บข้อความเอกสารที่ต้องการอ่าน

คอนสตรัคเตอร์ (Constructor)	
ParseEvent ()	สร้างอินสแตนซ์เพื่อทำการกำหนดค่าภายหลังหรือเป็นดีฟอลท์คอนสตรัคเตอร์(Default Constructor)

ฟังก์ชัน (Function)	
virtual const char*	getName () const=0
virtual const char*	getNamespace () const=0
virtual char*	getValue (const char* attrname) const=0
virtual int	getCountAttr() const=0
virtual const char*	getAttribute (int index) const=0
virtual char*	toString ()=0
virtual const char*	getText () const=0
virtual int	getType () const=0
virtual int	endCheck (ParseEvent* start)=0
virtual const char*	getError ()const=0

1.9 **คลาสแอตทริบิวต์ (Attribute Class)** เป็นคลาสที่ใช้ในการจัดการกับข้อมูลของเอ็็กซ์เอ็มแอล โดยทำหน้าที่ในแอตทริบิวต์ต่างๆของข้อมูลเอ็็กซ์เอ็มแอล

แอตทริบิวต์ (Attribute)	
Str	name เก็บชื่อของแอตทริบิวต์
Str	prefix เก็บค่าพรีฟิกของแอตทริบิวต์
Str	value เก็บค่าของแอตทริบิวต์

คอนสตรัคเตอร์ (Constructor)	
Attribute ()	สร้างอินสแตนซ์เพื่อทำการกำหนดค่าภายหลังหรือเป็นดีฟอลท์คอนสตรัคเตอร์(Default Constructor)

Attribute (const char* n, const char* v)	สร้างอินสแตนซ์โดยทำการกำหนดชื่อของแอตทริบิวต์จากพารามิเตอร์ const char* n และค่าของแอตทริบิวต์จาก const char* v
Attribute (const char* ns, const char* n, const char* v)	สร้างอินสแตนซ์โดยทำการกำหนดชื่อของแอตทริบิวต์จากพารามิเตอร์ const char* n, ค่าของเนมสเปซของแอตทริบิวต์จาก const char* ns และค่าของแอตทริบิวต์จาก const char* v
Attribute (const char *string)	สร้างอินสแตนซ์โดยทำการกำหนดจากข้อความแอตทริบิวต์ const char* string

ฟังก์ชัน (Function)	
const char*	getValue () const เป็นคอนสแตนต์ฟังก์ชันที่ใช้ในการเรียกดูค่าของแอตทริบิวต์ และส่งค่ากลับเป็น const char*
const char*	getName () const เป็นคอนสแตนต์ฟังก์ชันที่ใช้ในการเรียกดูชื่อของแอตทริบิวต์ และส่งค่ากลับเป็น const char*
const char*	getNamespace () const เป็นคอนสแตนต์ฟังก์ชันที่ใช้ในการเรียกดูเนมสเปซของแอตทริบิวต์ และส่งค่ากลับเป็น const char*
char*	toString () ใช้ในการเรียกดูข้อความแอตทริบิวต์ในรูปแบบเอกสารเอ็กซ์เอ็มแอล และส่งค่ากลับเป็น char*
char*	toPrefix_Name () ใช้ในการเรียกดูข้อความแอตทริบิวต์ในรูปแบบของพรีฟิกและชื่อของแอตทริบิวต์ซึ่งค้นด้วยเครื่องหมาย : และส่งค่ากลับเป็น char*

1.10 คลาสเอ็กซ์เอ็มแอล (Xml Class) เป็นคลาสที่ใช้ในการจัดการกับข้อมูลของเอ็กซ์เอ็มแอล โดยทำหน้าที่ในส่วนของการเอนโค้ด และ การดีโค้ดข้อมูลภาษาเอ็กซ์เอ็มแอล

แอตทริบิวต์ (Attribute)	
static int	ENCODE_128 ใช้เก็บค่าที่บอกลถึงการเข้ารหัสที่มีค่ามากกว่า 127
static const char	COMMENT ใช้เก็บค่าตัวอักษรที่แทนชนิดรูปแบบของคอมเมนต์
static const char	DOCTYPE ใช้เก็บค่าตัวอักษรที่แทนชนิดรูปแบบของการกำหนดโครงสร้างเอกสารเอ็กซ์เอ็มแอล
static const char	END_DOCUMENT ใช้เก็บค่าตัวอักษรที่แทนชนิดรูปแบบของคลาสเอ็นดอกรูเมนต์ตามโครงสร้างของเอกสารเอ็กซ์เอ็มแอล
static const char	END_TAG ใช้เก็บค่าตัวอักษรที่แทนชนิดรูปแบบของคลาสเอ็นเท็กตามโครงสร้างของเอกสารเอ็กซ์เอ็มแอล

static const char	END_DOCUMENT ใช้เก็บค่าตัวอักษรที่แทนชนิดรูปแบบของคลาสเอ็นดอคกุเมนต์ตามโครงสร้างของเอกสารเอ็กซ์เอ็มแอล
static const char	PROCESSING_INSTRUCTION ใช้เก็บค่าตัวอักษรที่แทนชนิดรูปแบบของโพรเซสซิงอินสตรักชันตามโครงสร้างของเอกสารเอ็กซ์เอ็มแอล
static const char	STARTTAG ใช้เก็บค่าตัวอักษรที่แทนชนิดรูปแบบของคลาสสตาร์ทแท็กตามโครงสร้างของเอกสารเอ็กซ์เอ็มแอล
static const int	ENCODE_MIN ใช้เก็บค่าที่บอกถึงรูปแบบของการเข้ารหัส
static const int	ENCODE_QUOT ใช้เก็บค่าที่บอกถึงการเข้ารหัสที่รวมไปถึงการเข้ารหัสในส่วน ของเครื่องหมาย “ ด้วย
static const char	LT ใช้เก็บค่าอักษรที่ใช้แทนเครื่องหมาย <
static const char	GT ใช้เก็บค่าอักษรที่ใช้แทนเครื่องหมาย >
static const char	AMP ใช้เก็บค่าอักษรที่ใช้แทนเครื่องหมาย &
static const char	QUOT ใช้เก็บค่าอักษรที่ใช้แทนเครื่องหมาย “
static const char	APOS ใช้เก็บค่าอักษรที่ใช้แทนเครื่องหมาย ‘
static const char	CDATA ใช้เก็บค่าอักษรที่ใช้แทนเครื่องหมาย #
static const char*	LT_S ใช้เก็บข้อความที่มีค่าเป็น “<”
static const char*	GT_S ใช้เก็บข้อความที่มีค่าเป็น “>”
static const char*	SL_S ใช้เก็บข้อความที่มีค่าเป็น “/”
static const char*	SP_S ใช้เก็บข้อความที่มีค่าเป็น “ ”
static const char*	PI_S ใช้เก็บข้อความที่มีค่าเป็น “?”
static const char*	CM_S ใช้เก็บข้อความที่มีค่าเป็น “;”
static const char*	CL_S ใช้เก็บข้อความที่มีค่าเป็น “:”
static const char*	QU_S ใช้เก็บข้อความที่มีค่าเป็น “”
static const char*	AD_S ใช้เก็บข้อความที่มีค่าเป็น “&”
static const char*	AP_S ใช้เก็บข้อความที่มีค่าเป็น “”
static const char*	ENTER_S ใช้เก็บข้อความที่มีค่าเป็น “\n”
static const char*	EQ_S ใช้เก็บข้อความที่มีค่าเป็น “=”
static const char*	SM_S ใช้เก็บข้อความที่มีค่าเป็น “;”

คอนสตรัคเตอร์ (Constructor)	
Xml()	สร้างอินสแตนซ์เพื่อทำการกำหนดค่าภายหลังหรือเป็นดีฟอลท์คอนสตรัคเตอร์(Default Constructor)

ฟังก์ชัน (Function)	
static char*	decode(const char* cooked) ใช้ในการถอดรหัสข้อความหากตัวอักษรตัวใดเป็นตัวเอ็นดีดีเรฟเฟอร์เรนที่จะทำการถอดรหัส โดยข้อความที่ต้องการจะถอดรหัสจะอยู่ในพารามิเตอร์ const char* cooked และส่งค่ากลับเป็น char* ซึ่งเป็นข้อความที่ถูกถอดรหัสแล้ว
static char*	encode (const char* raw) ใช้ในการเข้ารหัสข้อความหากตัวอักษรตัวใดเป็นตัวเอ็นดีดีเรฟเฟอร์เรนที่จะทำการเข้ารหัส โดยข้อความที่ต้องการจะเข้ารหัสจะอยู่ในพารามิเตอร์ const char* cooked และส่งค่ากลับเป็น char* ซึ่งเป็นข้อความที่ถูกเข้ารหัสแล้ว โดยจะทำการเรียกใช้ฟังก์ชัน encode (const char* raw, int flags) ที่มีค่า int flags เป็น ENCODE_MIN
static char*	encode (const char* raw, int flags) ใช้ในการเข้ารหัสข้อความหากตัวอักษรตัวใดเป็นตัวเอ็นดีดีเรฟเฟอร์เรนที่จะทำการเข้ารหัส โดยข้อความที่ต้องการจะเข้ารหัสจะอยู่ในพารามิเตอร์ const char* cooked และส่งค่ากลับเป็น char* ซึ่งเป็นข้อความที่ถูกเข้ารหัสแล้ว และยังสามารถที่จะใช้กำหนดรูปแบบของการเข้ารหัสได้จากพารามิเตอร์ int flags
static char*	quote (const char* q) จะทำการเรียกใช้ฟังก์ชัน encode (const char* raw) โดยทำการส่งพารามิเตอร์ const char* q เข้าไปเมื่อได้รับข้อความที่ได้ถูกเข้ารหัสเรียบร้อยแล้วจะทำการนำข้อความดังกล่าวใส่ไว้ภายในเครื่องหมาย “”

1.11 คลาสพรีฟิกแมพ (PrefixMap Class) เป็นคลาสที่ใช้ในการจัดการกับข้อมูลของเอ็กซ์เอ็มแอล โดยทำหน้าที่ในส่วนของพรีฟิก และ เนมสเปซข้อมูลภาษาเอ็กซ์เอ็มแอล

แอททริบิวต์ (Attribute)	
int	size ใช้เก็บขนาดของข้อมูลที่อยู่ในคลาส
char	prefixMap[SIZE][LEN] ใช้เก็บข้อความที่บอกถึงเนมสเปซที่สามารถหาจากพรีฟิกได้
char	namespaceMap[SIZE][LEN] ใช้เก็บข้อความที่บอกถึงพรีฟิกที่สามารถหาจากเนมสเปซได้

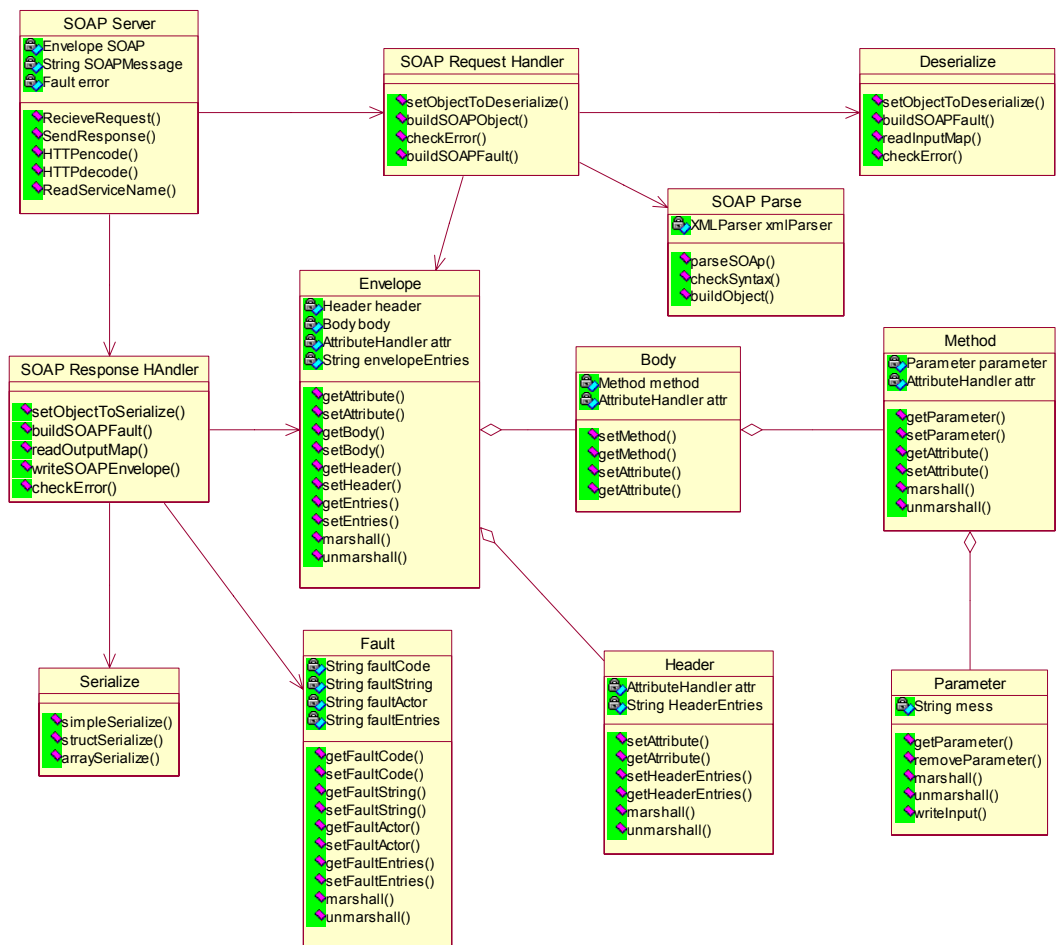
คอนสตรัคเตอร์ (Constructor)	
PrefixMap()	สร้างอินสแตนซ์เพื่อทำการกำหนดค่าภายหลังหรือเป็นดีฟอลท์คอนสตรัคเตอร์(Default Constructor)
PrefixMap (PrefixMap base,	สร้างอินสแตนซ์โดยทำการกำหนดค่าอินสแตนซ์ของคลาสพรีฟิกแม

const char* pf,const char* ns)	ปจากพารามิเตอร์ PrefixMap base, ค่าของพรีฟิกที่ต้องการเพิ่มเข้าไปจาก const char* pf, ค่าของเนมสเปซที่ต้องการเพิ่มเข้าไปจาก const char* ns
--------------------------------	---

ฟังก์ชัน (Function)	
PrefixMap&	Operator=(const PrefixMap& value) โอเวอร์โหลดโอเปอเรเตอร์เท่ากับที่ทำการก๊อปปี้อินสแตนซ์ของคลาสที่มีค่าของแอตทริบิวต์ภายในเท่ากับอินสแตนซ์ที่กำหนดจากพารามิเตอร์ const PrefixMap& value
int	add (const char* ns, const char* pf) ใช้ในการเพิ่มข้อมูลทั้งส่วนของเนมสเปซที่กำหนดจากพารามิเตอร์ const char* ns และพรีฟิกที่กำหนดจากพารามิเตอร์ const char* pf เข้าไปในคลาส และส่งค่ากลับเป็น 1 ถ้าสามารถเพิ่มข้อมูลได้ และส่งค่ากลับเป็น 0 ถ้าไม่ได้
int	add (const char* st) ใช้ในการเพิ่มข้อมูลทั้งส่วนของเนมสเปซ และพรีฟิกที่กำหนดจากพารามิเตอร์ const char* st เข้าไปในคลาสโดยจะทำการแยกส่วนทั้งสองออกจากข้อความ และส่งค่ากลับเป็น 1 ถ้าสามารถเพิ่มข้อมูลได้ และส่งค่ากลับเป็น 0 ถ้าไม่ได้
int	remove (const char* ns, const char* pf) ใช้ในการลบข้อมูลออกจากคลาสโดยข้อมูลที่ถูกลบจะต้องมีค่าเนมสเปซที่เหมือนกับพารามิเตอร์ const char* ns และมีค่าพรีฟิกเหมือนกับพารามิเตอร์ const char* pf และส่งค่ากลับเป็น 1 ถ้าสามารถเพิ่มข้อมูลได้ และส่งค่ากลับเป็น 0 ถ้าไม่ได้
const char*	getNamespace (const char* pf) const เป็นคอนสแตนต์ฟังก์ชันที่ใช้ในการหาค่าของเนมสเปซโดยหาจากพารามิเตอร์ const char* pf
const char*	getPrefix (const char* ns) const เป็นคอนสแตนต์ฟังก์ชันที่ใช้ในการหาค่าของพรีฟิกโดยหาจากพารามิเตอร์ const char* ns
const char*	getprefixMap (int i) const เป็นคอนสแตนต์ฟังก์ชันที่ใช้ในการหาค่าของพรีฟิกที่อ้างถึงโดยพารามิเตอร์ int i
const char*	getnsMap (int i) const เป็นคอนสแตนต์ฟังก์ชันที่ใช้ในการหาค่าของเนมสเปซที่อ้างถึงโดยพารามิเตอร์ int i
int	getSize () const เป็นคอนสแตนต์ฟังก์ชันที่ใช้ในการหาค่าของขนาดข้อมูลภายในคลาส

2. โชนพาร์เซอ์

ส่วนของโชนพาร์เซอร์เป็นส่วนที่ใช้ในการจัดการโชนแมสเสจ



รูป กลาสไคอะแกรมในส่วนของโซบพาร์เซอร์

2.1 คลาสเอ็นวิลอป (Envelope Class) เป็นคลาสที่ใช้แทนส่วนของเอ็นวิลอป ภายในโคมแมสเชจ ซึ่งจะมีแอตทริบิว คือ คลาสเฮดเดอร์, คลาสบอดี และ คลาสแอตทริบิวแฮนเดิลเลอร์ (AttributeHandler Class) ซึ่งจะมีฟังก์ชันในการกำหนดค่า และเรียกค่าคืนกลับจากคลาสที่เป็นสมาชิกส่วนตัวในคลาส ฟังก์ชันในการสร้างโคมแมสเชจ และฟังก์ชันในการนำโคมแมสเชจ มาสร้างเป็นคลาส

แอตทริบิวต์ (Attribute)	
Header	Header
Body	body เก็บอินสแตนซ์ของคลาส Body
Str	envelopeEntries เก็บค่าอีลิเมนต์ภายในของส่วน
AttributeHandler	attrHandler เก็บอินสแตนซ์ของคลาส AttributeHandler ซึ่งใช้จัดการแอตทริบิวต์ของส่วนเ็นวิลอป

Str	error เก็บข้อความที่บ่งบอกความผิดพลาดที่เกิดขึ้นกับอินสแตนซ์ของคลาสที่สร้างขึ้น
-----	---

คอนสตรัคเตอร์ (Constructor)	
Envelope()	สร้างอินสแตนซ์เพื่อทำการกำหนดค่าภายหลังหรือเป็นดีฟอลท์คอนสตรัคเตอร์(Default Constructor)
Envelope (const Body b, const char* envEntries)	สร้างอินสแตนซ์โดยทำการกำหนดค่าส่วนของอินสแตนซ์ของคลาส Body จากพารามิเตอร์ const Body b
Envelope(const Header h, const Body b,const char* envEntries)	สร้างอินสแตนซ์โดยทำการกำหนดค่าส่วนของอินสแตนซ์ของคลาส Header จากพารามิเตอร์ const Header h, คลาส Body จากพารามิเตอร์ const Body b และกำหนดส่วนของเอนวิลอปเอนทรีจากพารามิเตอร์ const char* envEntries
Envelope(const Header h, const Body b, const char* envEntries, const AttributeHandler attr)	สร้างอินสแตนซ์โดยทำการกำหนดค่าส่วนของอินสแตนซ์ของคลาส Header จากพารามิเตอร์ const Header h, คลาส Body จากพารามิเตอร์ const Body b, กำหนดส่วนของเอนวิลอปเอนทรีจากพารามิเตอร์ const char* envEntries และส่วนของคลาส AttributeHandler จากพารามิเตอร์ const AttributeHandler attr

ฟังก์ชัน (Function)	
Envelope&	operator=(const Envelope& value) โอเวอร์โหลดโอเปอเรเตอร์เท่ากับ ใช้ในการสร้างอินสแตนซ์ของคลาส Envelope โดยทำการก๊อปปี้ค่าภายในอินสแตนซ์ของคลาส Envelope ที่กำหนดจากพารามิเตอร์ const Envelope& value และส่งค่ากลับเป็น Envelope&
void	setAttribute(const char* attrQName, int value) ใช้ในการกำหนดค่าของแอตทริบิวต์ภายในคลาส AttributeHandler ที่อยู่ภายในคลาสซึ่งกำหนดข้อความแอตทริบิวต์จากพารามิเตอร์ const char* attrQName และค่าของแอตทริบิวต์จาก int value
const char*	getAttribute (QName attrQName) const ใช้ในการเรียกดูค่าของแอตทริบิวต์ที่มีข้อความแอตทริบิวต์เหมือนกับพารามิเตอร์ QName attrQName และส่งค่ากลับเป็น const char*
const AttributeHandler	getAttributeHandler () const ใช้ในการเรียกดูค่าอินสแตนซ์ของคลาส AttributeHandler ที่อยู่ภายในคลาส Envelope และส่งค่ากลับเป็น const AttributeHandler
char*	getAttribute() const ใช้ในการเรียกดูข้อความแอตทริบิวต์ทั้งหมดภายในคลาส และส่งค่ากลับเป็น char*

void	setHeader(Header h) ใช้ในการกำหนดค่าอินสแตนซ์ของคลาส Header จากพารามิเตอร์ Header h
const Header	getHeader () const ใช้ในการเรียกดูค่าอินสแตนซ์ของคลาส Header ที่อยู่ในคลาส Envelope และส่งค่ากลับเป็น const Header
void	setBody(Body b) ใช้ในการกำหนดค่าอินสแตนซ์ของคลาส Body ที่อยู่ในคลาส Envelope จากพารามิเตอร์ Body b
const Body	getBody() const ใช้ในการเรียกดูค่าอินสแตนซ์ของคลาส Body ที่อยู่ในคลาส Envelope และส่งค่ากลับเป็น const Body
void	setEnvelopeEntries(const char* envEntries) ใช้ในการกำหนดค่าเอ็นทรีภายในคลาส Envelope จากพารามิเตอร์ const char* envEntries
const char*	getEnvelopeEntries () const ใช้ในการเรียกดูค่าเอ็นทรีที่อยู่ในคลาส Envelope และส่งค่ากลับเป็น const char*
void	setError (const char* e) ใช้ในการกำหนดค่าของข้อความผิดพลาด ซึ่งกำหนดโดยพารามิเตอร์ const char* e ที่บอกถึงข้อความบอกความผิดพลาดที่ต้องการกำหนดให้กับแอตทริบิวต์ Str error ของคลาส
const char*	getError () const เป็นคอนสแตนต์ฟังก์ชันที่ใช้ในเรียกดูข้อความบอกความผิดพลาดของคลาสที่มีอยู่ปัจจุบัน และส่งค่ากลับเป็น const char*
int	getLength () const ใช้ในการเรียกดูค่าของขนาดความยาวของข้อมูลในส่วนของอินสแตนซ์ของคลาส Envelope
char*	marshall () ใช้ในการเปลี่ยนข้อมูลภายในคลาสให้อยู่ในรูปของโซบแมสเสจ
static Envelope*	unmarshall (Parser *pr, ParseEvent *pe) เป็นสตาดิกฟังก์ชันที่ใช้ในการเปลี่ยนจากโซบแมสเสจให้เป็นอินสแตนซ์ของคลาส Envelope

2.2 คลาสเฮดเดอร์ (Header Class) เป็นคลาสที่ใช้แทนส่วนหัวของโซบ (SOAP Header) ภายในโซบแมสเสจซึ่งจะมีแอตทริบิวต์ คือ คลาสพารามิเตอร์เฮดเดอร์เอนทรี (HeaderEntries Class) และ คลาสแอตทริบิวต์แฮนเดิลเลอร์ ซึ่งจะมีฟังก์ชันในการกำหนดค่า และเรียกค่าคืนกลับจากคลาสที่เป็นสมาชิกส่วนตัวในคลาส ฟังก์ชันในการสร้าง โซบแมสเสจ และฟังก์ชันในการนำโซบแมสเสจ มาสร้างเป็นคลาส

แอตทริบิวต์ (Attribute)	
Str	headerEntries เก็บค่าอีลิเมนต์ภายในของส่วน Header
AttributeHandler	attrHandler เก็บอินสแตนซ์ของคลาส AttributeHandler ซึ่งใช้จัดการแอตทริบิวต์ของส่วนเอ็นวิลอป
Str	error เก็บข้อความที่บ่งบอกความผิดพลาดที่เกิดขึ้นกับอินสแตนซ์ของคลาสที่สร้างขึ้น

คอนสตรัคเตอร์ (Constructor)	
Header()	สร้างอินสแตนซ์เพื่อทำการกำหนดค่าภายหลังหรือเป็นดีฟอลท์คอนสตรัคเตอร์(Default Constructor)
Header(const char* headerEn)	สร้างอินสแตนซ์โดยทำการกำหนดค่าอีลิเมนต์ภายในของส่วน headerEntries ซึ่งกำหนดจากพารามิเตอร์ const char* headerEn
Header(const char* headerEn, const AttributeHandler attr)	สร้างอินสแตนซ์โดยทำการกำหนดค่าอีลิเมนต์ภายในของส่วน headerEntries ซึ่งกำหนดจากพารามิเตอร์ const char* headerEn และค่าของ attrHandler ซึ่งกำหนดจากพารามิเตอร์ const AttributeHandler attr

ฟังก์ชัน (Function)	
Header&	operator=(const Header& that) โอเวอร์โหลดโอเปอเรเตอร์เท่ากับใช้ในการสร้างอินสแตนซ์ของคลาส Header โดยทำการก๊อปปี้ค่าภายในอินสแตนซ์ของคลาส Header ที่กำหนดจากพารามิเตอร์ const Header& value และส่งค่ากลับเป็น Header&
void	setAttribute(const char* attrQName, int value) ใช้ในการกำหนดค่าของแอตทริบิวต์ภายในคลาส AttributeHandler ที่อยู่ภายในคลาสซึ่งกำหนดข้อความแอตทริบิวต์จากพารามิเตอร์ const char* attrQName และค่าของแอตทริบิวต์จาก int value
const char*	getAttribute (QName attrQName) const ใช้ในการเรียกดูค่าของแอตทริบิวต์ที่มีข้อความแอตทริบิวต์เหมือนกับพารามิเตอร์ QName attrQName และส่งค่ากลับเป็น const char*
const AttributeHandler	getAttributeHandler () const ใช้ในการเรียกดูค่าอินสแตนซ์ของคลาส AttributeHandler ที่อยู่ภายในคลาส Envelope และส่งค่ากลับเป็น const AttributeHandler
char*	getAttribute() const ใช้ในการเรียกดูข้อความแอตทริบิวต์ทั้งหมดภายในคลาส และส่งค่ากลับเป็น char*
void	setHeaderEntries(const char* headerEn)ใช้ในการกำหนดค่าเอ็นทรีภายในคลาส Envelope จากพารามิเตอร์ const char* headerEn
const char*	getHeaderEntries () const ใช้ในการเรียกดูค่าเอ็นทรีที่อยู่ภายในคลาส Envelope และส่งค่ากลับเป็น const char*
void	setError (const char* e) ใช้ในการกำหนดค่าของข้อความผิดพลาด ซึ่งกำหนดโดยพารามิเตอร์ const char* e ที่บอกถึงข้อความบอกความผิดพลาดที่ต้องการกำหนดให้กับแอตทริบิวต์ Str error ของคลาส

const char*	getError () const เป็นคอนสแตนต์ฟังก์ชันที่ใช้ในเรียกดูข้อความบอกความผิดพลาดของคลาสที่มีอยู่ปัจจุบัน และส่งค่ากลับเป็น const char*
int	getLength () const ใช้ในการเรียกดูค่าของขนาดความยาวของข้อมูลในส่วนของอินสแตนท์ของคลาส Header
char*	marshall () ใช้ในการเปลี่ยนข้อมูลภายในคลาสให้อยู่ในรูปของโซบแมสเสจ
static Header*	unmarshall (Parser *pr,ParseEvent *pe) เป็นสแตติกฟังก์ชันที่ใช้ในการเปลี่ยนจากโซบแมสเสจให้เป็นอินสแตนท์ของคลาส Header และส่งค่ากลับเป็น Header*

2.3 คลาสบอดี (Body Class) เป็นคลาสที่ใช้แทนส่วนเนื้อหาของโซบ (SOAP Body) ภายในโซบแมสเสจ ซึ่งจะมีแอททริบิวต์ คือ คลาส Method และ คลาสแอททริบิวต์แฮนเดิลเลอร์ ซึ่งจะมีฟังก์ชันในการกำหนดค่า และเรียกค่าคืนกลับจากคลาสที่เป็นสมาชิกส่วนตัวในคลาส ฟังก์ชันในการสร้างโซบแมสเสจ และฟังก์ชันในการนำโซบแมสเสจ มาสร้างเป็นคลาส

แอททริบิวต์ (Attribute)	
Method	method เก็บค่าอินสแตนท์ของคลาส Method ที่อยู่ภายในคลาส Body
Str	bodyEntries เก็บค่าเอ็นทรีภายในของคลาส Body
Str	error เก็บข้อความที่บ่งบอกความผิดพลาดที่เกิดขึ้นกับอินสแตนท์ของคลาสที่สร้างขึ้น

คอนสตรัคเตอร์ (Constructor)	
Body()	สร้างอินสแตนท์เพื่อทำการกำหนดค่าภายหลังหรือเป็นดีฟอลท์คอนสตรัคเตอร์(Default Constructor)
Body(const char* bEntries)	สร้างอินสแตนท์โดยทำการกำหนดค่าอีลิเมนต์ภายในของส่วน headerEntries ซึ่งกำหนดจากพารามิเตอร์ const char* bEntries

ฟังก์ชัน (Function)	
Body&	operator=(const Body& that) โอเวอร์โหลดโอเปอเรเตอร์เท่ากับใช้ในการสร้างอินสแตนท์ของคลาส Body โดยทำการก๊อปปี้ค่าภายในอินสแตนท์ของคลาส Body ที่กำหนดจากพารามิเตอร์ const Body& value และส่งค่ากลับเป็น Body&
void	setBodyEntries(const char* bEntries) ใช้ในการกำหนดค่าเอ็นทรีภายในคลาส Body จากพารามิเตอร์ const char* bEntries
const char*	getBodyEntries () const ใช้ในการเรียกดูค่าเอ็นทรีที่อยู่ภายในคลาส

	Body และส่งค่ากลับเป็น const char*
int	getLength () const ใช้ในการเรียกดูค่าของขนาดความยาวของข้อมูลในส่วนของอินสแตนซ์ของคลาส Body
void	setMethod(const Method m1) ใช้ในการกำหนดค่าของอินสแตนซ์ของคลาส Method ซึ่งกำหนดจากพารามิเตอร์ const Method m1
const Method	getMethod () ใช้ในการเรียกดูอินสแตนซ์ของคลาส Method และส่งค่ากลับเป็น const Method
char*	marshall () ใช้ในการเปลี่ยนข้อมูลภายในคลาสให้อยู่ในรูปของโซบแมสเสจ และส่งค่ากลับเป็น char*
static Body*	unmarshall (Parser* pr,ParseEvent *pe) เป็นสตาดิกฟังก์ชันที่ใช้ในการเปลี่ยนจากโซบแมสเสจให้เป็นอินสแตนซ์ของคลาส Header และส่งค่ากลับเป็น Body*

2.4 คลาสเมทอด (Method Class) เป็นคลาสที่ใช้แทนส่วนเมทอดของส่วนเนื้อหาของโซบแมสเสจ ซึ่งจะมีแอททริบิวต์ คือ คลาสเมทอด และ คลาสแอททริบิวต์แฮนเดิลเลอร์ ซึ่งจะมีฟังก์ชันในการกำหนดค่า และเรียกค่าคืนกลับจากคลาสที่เป็นสมาชิกส่วนตัวในคลาส ฟังก์ชันในการสร้างโซบแมสเสจ และฟังก์ชันในการนำโซบแมสเสจ มาสร้างเป็นคลาส

แอททริบิวต์ (Attribute)	
Str	name เก็บชื่อของเมทอด
Str	url เก็บค่ายูอาร์แอล
Str	encoding เก็บค่าเอ็นโค้ดดิ้งสไตล์ (EncodingStyle)
Str	error เก็บข้อความที่บ่งบอกความผิดพลาดที่เกิดขึ้นกับอินสแตนซ์ของคลาสที่สร้างขึ้น
Parameter	parameter เก็บค่าอินสแตนซ์ของคลาส Parameter

คอนสตรัคเตอร์ (Constructor)	
Method()	สร้างอินสแตนซ์เพื่อทำการกำหนดค่าภายหลังหรือเป็นดีฟอลท์คอนสตรัคเตอร์(Default Constructor)

ฟังก์ชัน (Function)	
Method&	operator=(const Method& that) โอเวอร์โหลดโอเปอเรเตอร์เท่ากับใช้ในการสร้างอินสแตนซ์ของคลาส Method โดยทำการก๊อปปี้ค่าภายในอินสแตนซ์ของคลาส Method ที่กำหนดจากพารามิเตอร์ const Method & that และส่งค่ากลับเป็น Method &

int	getLength () const ใช้ในการเรียกดูค่าของขนาดความยาวของข้อมูลในส่วนของอินสแตนท์ของคลาส Method
void	setParameter (const char* p) ใช้ในการกำหนดค่าของอินสแตนท์ของคลาส Parameter ซึ่งกำหนดจากพารามิเตอร์ const char* p
const Parameter	getParameter () ใช้ในการเรียกดูอินสแตนท์ของคลาส Parameter และส่งค่ากลับเป็น const Parameter
void	setMethodName(const char* m) ใช้ในการกำหนดค่าของชื่อเมธอดของคลาสซึ่งกำหนดจากพารามิเตอร์ const char* m
const char*	getMethodName () ใช้ในการเรียกดูค่าของชื่อเมธอดของคลาส และส่งค่ากลับเป็น const char*
void	setMethodURL(const char* u) ใช้ในการกำหนดค่าของ url ของคลาสซึ่งกำหนดจากพารามิเตอร์ const char* u
const char*	getMethodURL () ใช้ในการเรียกดูค่าของ url ของคลาส และส่งค่ากลับเป็น const char*
void	setMethodEnc(const char* e) ใช้ในการกำหนดค่า encodingStyle ของคลาสซึ่งกำหนดจากพารามิเตอร์ const char* e
const char*	getMethodEnc () ใช้ในการเรียกดูค่า encodingStyle ของคลาส และส่งค่ากลับเป็น const char*
void	setError (const char* e) ใช้ในการกำหนดค่าของข้อความผิดพลาด ซึ่งกำหนดโดยพารามิเตอร์ const char* e ที่บอกถึงข้อความบอกความผิดพลาดที่ต้องการกำหนดให้กับแอตทริบิวต์ Str error ของคลาส
const char*	getError () const เป็นคอนสแตนท์ฟังก์ชันที่ใช้ในการเรียกดูข้อความบอกความผิดพลาดของคลาสที่มีอยู่ปัจจุบัน และส่งค่ากลับเป็น const char*
int	getLength () const ใช้ในการเรียกดูค่าของขนาดความยาวของข้อมูลในส่วนของอินสแตนท์ของคลาส Method
char*	marshall () ใช้ในการเปลี่ยนข้อมูลภายในคลาสให้อยู่ในรูปแบบของไชบแมสเสจ และส่งค่ากลับเป็น char*
static Method*	unmarshall (Parser* pr, ParseEvent *pe) เป็นสตาดิกฟังก์ชันที่ใช้ในการเปลี่ยนจากไชบแมสเสจให้เป็นอินสแตนท์ของคลาส Header และส่งค่ากลับเป็น Method*

2.5 คลาสพารามิเตอร์ (Parameter Class) เป็นคลาสที่ใช้แทนส่วนพารามิเตอร์ภายในคลาสเมธอด ซึ่งจะมีแอตทริบิวต์ คือ คลาสพารามิเตอร์เอนตริ (ParameterEntries Class) และ คลาสแอตทริบิวต์เอนเดอร์

เลอร์ ซึ่งจะมีฟังก์ชันในการกำหนดค่า และเรียกค่าคืนกลับจากคลาสที่เป็นสมาชิกส่วนตัวในคลาส ฟังก์ชันในการสร้างโชนแมสเซจ และฟังก์ชันในการนำโชนแมสเซจ มาสร้างเป็นคลาส

แอททริบิวต์ (Attribute)	
Str	params เก็บข้อมูลพารามิเตอร์ภายในคลาส
Str	error เก็บข้อความที่บ่งบอกความผิดพลาดที่เกิดขึ้นกับอินสแตนซ์ของคลาสที่สร้างขึ้น

คอนสตรัคเตอร์ (Constructor)	
Parameter()	สร้างอินสแตนซ์เพื่อทำการกำหนดค่าภายหลังหรือเป็นดีฟอลท์คอนสตรัคเตอร์(Default Constructor)
Parameter(const char* pa)	สร้างอินสแตนซ์โดยการกำหนดค่าของข้อมูลพารามิเตอร์จากพารามิเตอร์ const char* pa

ฟังก์ชัน (Function)	
Parameter&	operator=(const Parameter& that) โอเวอร์โหลดโอเปอเรเตอร์เท่ากับ ใช้ในการสร้างอินสแตนซ์ของคลาส Parameter โดยทำการก๊อปปี้ค่าภายในอินสแตนซ์ของคลาส Parameter ที่กำหนดจากพารามิเตอร์ const Parameter & that และส่งค่ากลับเป็น Parameter &
int	getLength () const ใช้ในการเรียกดูค่าของขนาดความยาวของข้อมูลในส่วนของอินสแตนซ์ของคลาส Parameter
void	setParameter(const char* p) ใช้ในการกำหนดค่าของข้อมูลพารามิเตอร์ภายในคลาสจาก const char* p
const char*	getParameter () ใช้ในการเรียกดูค่าของพารามิเตอร์ภายในคลาส และส่งค่ากลับเป็น const char*
void	setError (const char* e) ใช้ในการกำหนดค่าของข้อความผิดพลาด ซึ่งกำหนดโดยพารามิเตอร์ const char* e ที่บอกถึงข้อความบอกความผิดพลาดที่ต้องการกำหนดให้กับแอททริบิวต์ Str error ของคลาส
const char*	getError () const เป็นคอนสแตนซ์ฟังก์ชันที่ใช้ในเรียกดูข้อความบอกความผิดพลาดของคลาสที่มีอยู่ปัจจุบัน และส่งค่ากลับเป็น const char*
char*	marshall ()ใช้ในการเปลี่ยนข้อมูลภายในคลาสให้อยู่ในรูปแบบของโชนแมสเซจ และส่งค่ากลับเป็น char*

2.6 คลาสฟอลต์ (Fault Class) เป็นคลาสที่ใช้เก็บค่าความผิดพลาดต่าง ๆ ได้แก่ ใ้ค้บอกความผิดพลาด, ข้อความบอกความผิดพลาด, ข้อความระบุสิ่งที่ทำให้เกิดความผิดพลาด, เอลเลเมนต์ภายในที่ใช้ระบุ

ความผิดพลาด ซึ่งจะมีฟังก์ชันที่ใช้ในการกำหนดค่าให้กับสมาชิกของคลาส และเรียกค่าคืนกลับจากสมาชิกของคลาส และฟังก์ชันที่ใช้ในการสร้างโซบแมสเซจ

แอตทริบิวต์ (Attribute)	
Str	faultCode เก็บข้อความที่บอกถึง Fault Code
Str	faultString เก็บข้อความที่บอกถึง Fault String
Str	faultActorURI เก็บข้อความที่บอกถึง Fault ActorURI
Str	detailEntries เก็บข้อความที่บอกถึง Detail Entries
Str	faultEntries เก็บข้อความที่บอกถึง Fault Entries
AttributeHandler	attrHandler เก็บอินสแตนซ์ของคลาสที่ใช้ในการจัดเรแอตทริบิวต์

คอนสตรัคเตอร์ (Constructor)	
Fault()	สร้างอินสแตนซ์เพื่อทำการกำหนดค่าภายหลังหรือเป็นดีฟอลท์คอนสตรัคเตอร์(Default Constructor)
Fault(const char* fc, const char* fs)	สร้างอินสแตนซ์โดยการกำหนดค่า faultCode จากพารามิเตอร์ const char* fc และกำหนดค่าของ faultString จากพารามิเตอร์ const char* fs

ฟังก์ชัน (Function)	
Fault&	operator=(const Fault& that) โอเวอร์โหลดโอเปอเรเตอร์เท่ากับใช้ในการสร้างอินสแตนซ์ของคลาส Fault โดยทำการก๊อปปี้ค่าภายในอินสแตนซ์ของคลาส Fault ที่กำหนดจากพารามิเตอร์ const Fault & that และส่งค่ากลับเป็น Fault &
void	setAttribute(const char* attrQName, int value) ใช้ในการกำหนดค่าของแอตทริบิวต์ภายในคลาส AttributeHandler ที่อยู่ภายในคลาสซึ่งกำหนดข้อความแอตทริบิวต์จากพารามิเตอร์ const char* attrQName และค่าของแอตทริบิวต์จาก int value
const char*	getAttribute (QName attrQName) const ใช้ในการเรียกดูค่าของแอตทริบิวต์ที่มีข้อความแอตทริบิวต์เหมือนกับพารามิเตอร์ QName attrQName และส่งค่ากลับเป็น const char*
const AttributeHandler	getAttributeHandler () const ใช้ในการเรียกดูค่าอินสแตนซ์ของคลาส AttributeHandler ที่อยู่ภายในคลาส Envelope และส่งค่ากลับเป็น const AttributeHandler
void	setFaultCode(const char* fc) ใช้ในการกำหนดค่า FaultCode ของอินสแตนซ์ของคลาส Fault จาก const char* fc

const char*	getFaultCode() ใช้ในการเรียกดูค่าของ FaultCode ของอินสแตนซ์ของคลาส Fault และส่งค่ากลับเป็น const char*
void	setFaultString (const char* fs) ใช้ในการกำหนดค่า FaultString ของอินสแตนซ์ของคลาส Fault จาก const char* fs
const char*	getFaultString () ใช้ในการเรียกดูค่าของ FaultString ของอินสแตนซ์ของคลาส Fault และส่งค่ากลับเป็น const char*
void	setFaultActorURI (const char* faURI) ใช้ในการกำหนดค่า FaultActorURI ของอินสแตนซ์ของคลาส Fault จาก const char* faURI
const char*	getFaultActorURI () ใช้ในการเรียกดูค่าของ FaultActorURI ของอินสแตนซ์ของคลาส Fault และส่งค่ากลับเป็น const char*
void	setDetailEntries(const char* dE) ใช้ในการกำหนดค่า DetailEntries ของอินสแตนซ์ของคลาส Fault จาก const char* dE
const char*	getDetailEntries () ใช้ในการเรียกดูค่าของ DetailEntries ของอินสแตนซ์ของคลาส Fault และส่งค่ากลับเป็น const char*
void	setFaultEntries (const char* fe) ใช้ในการกำหนดค่า FaultEntries ของอินสแตนซ์ของคลาส Fault จาก const char* fe
const char*	getFaultEntries () ใช้ในการเรียกดูค่า FaultEntries ของอินสแตนซ์ของคลาส และส่งค่ากลับเป็น const char*
int	getLength () const ใช้ในการเรียกดูค่าของขนาดความยาวของข้อมูลในส่วนของอินสแตนซ์ของคลาส Fault
char*	marshall () ใช้ในการเปลี่ยนข้อมูลภายในคลาสให้อยู่ในรูปของโซบแมสเสจ
static Fault*	unmarshall(const char* fc,const char* fs) เป็นสตาดิกฟังก์ชันที่ใช้ในการเปลี่ยนจากโซบแมสเสจให้เป็นอินสแตนซ์ของคลาส Fault โดยทำการสร้างอินสแตนซ์ของคลาสแล้วกำหนดค่าในส่วนของ FaultCode จาก const char* fc และส่วนของ FaultString จาก const char* fs

2.7 คลาสแอททริบิวต์แฮนเดิลเลอร์ (AttributeHandler Class) เป็นคลาสที่ใช้ในการจัดการกับส่วนของแอททริบิวต์ ซึ่งจะมีฟังก์ชันที่ใช้ในการเรียกขอค่าของแอททริบิวต์ และฟังก์ชันเรียกขอค่านามสเปซจากค่าของส่วนนำหน้า (Prefix)

แอททริบิวต์ (Attribute)	
Str	attributes[4] เก็บข้อความแอตทริบิวต์ในรูปของอาร์เรย์
int	size เก็บขนาดของข้อมูล

คอนสตรัคเตอร์ (Constructor)	
AttributeHandler ()	สร้างอินสแตนซ์เพื่อทำการกำหนดค่าภายหลังหรือเป็นดีฟอลต์คอนสตรัคเตอร์(Default Constructor)
AttributeHandler(const char* attr,int s)	สร้างอินสแตนซ์โดยการกำหนดค่าข้อความแอตทริบิวต์จาก const cahr* attr และขนาดของข้อมูลจาก int s

ฟังก์ชัน (Function)	
AttributeHandler&	operator=(const AttributeHandler& that) โอเวอร์โหลดโอเปอเรเตอร์ที่ใช้ในการก๊อปปี้อินสแตนซ์ของคลาส AttributeHandler ที่กำหนดในส่วนของพารามิเตอร์ const AttributeHandler& that
void	setAttribute (const char* a, int sizeA) ใช้ในการกำหนดค่าข้อความแอตทริบิวต์ภายในและขนาดของข้อมูลจากพารามิเตอร์ const char* a และ int sizeA
int	getSize () const ใช้ในการเรียกดูขนาดของข้อมูลของอินสแตนซ์ของคลาส AttributeHandler และส่งค่ากลับเป็น int
char*	getAttribute() const ใช้ในการเรียกดูข้อความแอตทริบิวต์ทั้งหมดภายในคลาส และส่งค่ากลับเป็น char*
const char*	getAttribute (QName attrQName) const ใช้ในการเรียกดูค่าของแอตทริบิวต์ที่มีข้อความแอตทริบิวต์เหมือนกับพารามิเตอร์ QName attrQName และส่งค่ากลับเป็น const char*
char*	getValue(const char* attrName) const ใช้ในการเรียกดูค่าของแอตทริบิวต์ที่มีชื่อเหมือนกับพารามิเตอร์ const char* attrName และส่งค่ากลับเป็น char*
const char*	getAttribute (int index) const ใช้ในการเรียกดูค่าแอตทริบิวต์ ในลำดับที่กำหนดในส่วนของพารามิเตอร์ int index และส่งค่ากลับเป็น const char*
int	getLength () const ใช้เรียกดูขนาดความยาวของข้อมูลทั้งหมดภายในคลาส และส่งค่ากลับเป็น int
char*	marshall () ใช้ในการเปลี่ยนข้อมูลภายในคลาสให้อยู่ในรูปของไซบแมสเสจ
static AttributeHandler*	unmarshall(const char* src,int s) เป็นสตาดิกฟังก์ชันที่ใช้ในการเปลี่ยนจากไซบแมสเสจให้เป็นอินสแตนซ์ของคลาส AttributeHandler โดยทำการสร้างอินสแตนซ์ของคลาสแล้วกำหนดค่าในส่วนข้อความแอตทริบิวต์จาก const char* src และส่วนของขนาดข้อมูล int s และส่งค่า

	กลับ
--	------

2.8 คลาสคอนสแตนต์ (Constants Class) เป็นคลาสที่ใช้เก็บค่าคงที่ต่าง ๆ ที่จะนำมาใช้ภายใน SOAP Message

แอททริบิวต์ (Attribute)	
static const char*	XMLNS เก็บข้อความ “xmlns”
static const char*	SOAP เก็บข้อความ “SOAP”
static const char*	SOAP_ENV เก็บข้อความ “SOAP:ENV”
static const char*	SOAP_ENC เก็บข้อความ “SOAP:ENC”
static const char*	XSI เก็บข้อความ “xsi”
static const char*	XSD เก็บข้อความ “xsd”
static const char*	XMLNS_URI เก็บข้อความ “http://www.w3.org/2000/xmlns/”
static const char*	SOAP_ENV_URI เก็บข้อความ “http://schemas.xmlsoap.org/soap/envelope/”
static const char*	SOAP_ENC_URI เก็บข้อความ “http://schemas.xmlsoap.org/soap/encoding/”
static const char*	XSI1999_URI เก็บข้อความ “http://www.w3.org/1999/XMLSchema-instance”
static const char*	XSD1999_URI เก็บข้อความ “http://www.w3.org/1999/XMLSchema”
static const char*	XSI2000_URI เก็บข้อความ “http://www.w3.org/2000/10/XMLSchema-instance”
static const char*	XSD2000_URI เก็บข้อความ “http://www.w3.org/2000/10/XMLSchema”
static const char*	XSI2001_URI เก็บข้อความ “http://www.w3.org/2001/XMLSchema-instance”
static const char*	XSD2001_URI เก็บข้อความ “http://www.w3.org/2001/XMLSchema”
static const char*	XML_SOAP เก็บข้อความ “http://xml.apache.org/xml-soap”
static const char*	XML_SOAP_DEPLOYMENT เก็บข้อความ “http://xml.apache.org/xml-soap/deployment”
static const char*	LITERAL_XML เก็บข้อความ “http://xml.apache.org/xml-soap/literalxml”
static const char*	XMI_ENC เก็บข้อความ “http://www.ibm.com/namespaces/xmi”
static const char*	POST เก็บข้อความ “POST”
static const char*	HOST เก็บข้อความ “Host”
static const char*	CONTENT_TYPE เก็บข้อความ “Content-Type”
static const char*	CONTENT_TYPE_JMS เก็บข้อความ “ContentType”
static const char*	CONTENT_LENGTH เก็บข้อความ “Content-Length”
static const char*	CONTENT_LOCATION เก็บข้อความ “Content-Location”
static const char*	CONTENT_ID เก็บข้อความ “Content-ID”
static const char*	SOAP_ACTION เก็บข้อความ “SOAPAction”

static const char*	AUTHORIZATION เก็บข้อความ “Authorization”
static const char*	PROXY_AUTHORIZATION เก็บข้อความ “Proxy-Authorization”
static const char*	DEFAULT_CHARSET เก็บข้อความ “iso-8859-1”
static const char*	CHARSET_UTF8 เก็บข้อความ “utf-8”
static const char*	CONTENT_TYPE_V เก็บข้อความ “text/xml”
static const char*	CONTENT_TYPE_UTF8 เก็บข้อความ “text/xml; charset=utf-8”
static const char*	XML_DECL เก็บข้อความ “<?xml version='1.0' encoding='UTF-8'?>\r\n”
static const char*	ENVELOPE เก็บข้อความ “Envelope”
static const char*	BODY เก็บข้อความ “Body”
static const char*	HEADER เก็บข้อความ “Header”
static const char*	FAULT เก็บข้อความ “Fault”
static const char*	FAULT_CODE เก็บข้อความ “faultcode”
static const char*	FAULT_STRING เก็บข้อความ “faultstring”
static const char*	FAULT_ACTOR เก็บข้อความ “faultactor”
static const char*	DETAIL เก็บข้อความ “detail”
static const char*	FAULT_DETAIL_ENTRY เก็บข้อความ “detailEntry”
static const char*	PARAMETER เก็บข้อความ “Parameter”
static const char*	RETURN เก็บข้อความ “return”
static const char*	RESPONSE เก็บข้อความ “Response”
static const char*	NS เก็บข้อความ “ns”
static const char*	ENCODING_STYLE เก็บข้อความ “encodingStyle”
static const char*	MUST_UNDERSTAND เก็บข้อความ “mustUnderstand”
static const char*	TYPE เก็บข้อความ “type”
static const char*	A_NULL เก็บข้อความ “null”
static const char*	NIL เก็บข้อความ “nil”
static const char*	ARRAY_TYPE เก็บข้อความ “arraytype”
static const char*	REFERENCE เก็บข้อความ “href”
static const char*	ID เก็บข้อความ “id”
static const char*	TRUE เก็บข้อความ “true”
static const char*	VERSION_MISMATCH เก็บข้อความ “VersionMismatch”
static const char*	MUST_UNDERSTAND_F เก็บข้อความ “MustUnderstand”
static const char*	CLIENT เก็บข้อความ “Client”
static const char*	SERVER เก็บข้อความ “Server”

static const char*	PROTOCOL เก็บข้อความ “Protocol”
static const char*	BAD_TARGET_OBJECT_URI เก็บข้อความ “.BadTargetObjectURI”
static const char*	ERR_MSG_VERSION_MISMATCH เก็บข้อความ “SOAP-ENV:VersionMismatch: Envelope element must be associated with the 'SOAP-ENV' namespace.”
static const char*	ERR_BODYTAG เก็บข้อความ “Body must have in Envelope”

2.9 คลาสซีเรียไลเซอร์ (Serializer Class) เป็นคลาสที่ใช้ในการแปลงชนิดของข้อมูลให้เป็นค่าที่ถูกต้อง กำหนดรูปแบบภายในโซบแมสเสจ ตามข้อกำหนดในส่วนของการเข้ารหัสของโซบ ซึ่งจะมีฟังก์ชันที่ใช้ในการแปลงชนิดจากชนิดของข้อมูลพื้นฐาน, ชนิดข้อมูลแบบอะเรย์ และชนิดข้อมูลแบบคลาสหรือโครงสร้าง ให้เป็นรูปแบบของการเข้ารหัสของโซบ

แอททริบิวต์ (Attribute)	
Fault	fault เก็บค่าอินสแตนซ์ของคลาส Fault เพื่อบอกความผิดพลาดในการเปลี่ยนข้อมูลที่ได้จากเซอร์วิสโค้ดให้อยู่ในร
Str	SOAPMessage เก็บโซบแมสเสจเฉพาะส่วนของข้อมูลที่ได้จากการ

คอนสตรัคเตอร์ (Constructor)	
SOAPSerializer()	สร้างอินสแตนซ์เพื่อทำการกำหนดค่าภายหลังหรือเป็นดีฟอลท์คอนสตรัคเตอร์(Default Constructor)

ฟังก์ชัน (Function)	
Fault	buildSOAPFault () ใช้ในการสร้างอินสแตนซ์ของคลาส Fault ที่บอกความผิดพลาดในการเปลี่ยนข้อมูลที่ได้จากเซอร์วิสโค้ด และส่งค่ากลับเป็น Fault
char*	serialize () ใช้ในการสร้างโซบแมสเสจเฉพาะส่วนของข้อมูลที่ได้จากการอ่านเท็กซ็ไฟล์ที่ได้จากการทำงานของเซอร์วิสโค้ด และส่งค่ากลับเป็น char*

2.10 คลาสดีซีเรียไลเซอร์ (Deserializer Class) เป็นคลาสที่ใช้ในการแปลงค่าที่ถูกกำหนดรูปแบบภายใน โซบแมสเสจตามข้อกำหนดในส่วนของการเข้ารหัสของโซบให้เป็นชนิดของข้อมูล ซึ่งจะมีฟังก์ชันที่ใช้ในการแปลงรูปแบบของการเข้ารหัสของโซบ ให้เป็นชนิดจากชนิดของข้อมูลพื้นฐาน, ชนิดข้อมูลแบบอะเรย์ และชนิดข้อมูลแบบคลาสหรือโครงสร้าง

แอททริบิวต์ (Attribute)

Str	ServiceMethod เก็บค่าของเซอร์วิสเมธอด
Str	ServiceEncoding เก็บค่าของเซอร์วิสเอ็นโค้ดดิ้งสไคล์
Str	ServiceOutput เก็บค่าของเซอร์วิสเอาต์พุต
Str	ServiceQname เก็บค่าของเซอร์วิสเมธอดคิวเนม
Str	ServiceQnameNS เก็บค่าของเซอร์วิสเนมสเปซของคิวเนม
Str	ServiceParameter เก็บค่าของพารามิเตอร์ภายในเซอร์วิส
Str	ServiceMap เก็บค่าของการกำหนดแอตทริบิวต์ภายในคลาสที่ต้องทำการสร้างขึ้น
Str	error เก็บข้อความที่บ่งบอกความผิดพลาดที่เกิดขึ้นกับอินสแตนซ์ของคลาสที่สร้างขึ้น
static const char*	SERVICE เก็บข้อความ “service”
static const char*	NAMESPACEID เก็บข้อความ “namespaceid”
static const char*	INPUT เก็บข้อความ “input”
static const char*	OUTPUT เก็บข้อความ “output”
static const char*	PROVIDER เก็บข้อความ “provider”
static const char*	MAPPING เก็บข้อความ “mapping”
static const char*	MAP เก็บข้อความ “map”
static const char*	XMLNS_X เก็บข้อความ “xmlns:x”
static const char*	QNAME_A เก็บข้อความ “qname”
static const char*	HEADERFILE เก็บข้อความ “headerfile”
static const char*	XSI_TYPE เก็บข้อความ “xsi:type”
static const char*	INT_S เก็บข้อความ “int”
static const char*	DOUBLE_S เก็บข้อความ “double”
static const char*	LONG_S เก็บข้อความ “long”
static const char*	FLOAT_S เก็บข้อความ “float”
static const char*	STRING_S เก็บข้อความ “string”
static const char*	CHAR_S เก็บข้อความ “char”

คอนสตรัคเตอร์ (Constructor)	
SOAPDeserialize()	สร้างอินสแตนซ์เพื่อทำการกำหนดค่าภายหลังหรือเป็นดีฟอลท์คอนสตรัคเตอร์(Default Constructor)

ฟังก์ชัน (Function)	
void	readServiceMap(const char* fileName) ใช้ในการอ่านเท็กไฟล์ที่อธิบาย

	รายละเอียดของเซอร์วิสเพื่อทำการดึงรายละเอียดต่าง ๆ มาทำการกำหนดให้กับค่าภายในคลาส
int	isNative(const char* s) ตรวจสอบว่าเป็นชนิดข้อมูลพื้นฐานหรือไม่
char*	serialOutput () ใช้ในการเปลี่ยนลำดับชั้นของข้อมูลให้อยู่ในรูปของข้อความที่เรียงต่อกัน โดยมีชื่อของข้อมูลและค่าของ
static int	checkOrder (const char* fileName, const char* params)
void	setServiceMethod (const char* mname) ใช้ในการกำหนดค่าเซอร์วิสเมธอดจากพารามิเตอร์ const char* mname
const char*	getServiceMethod () const ใช้ในการเรียกดูค่าของเซอร์วิสเมธอด และส่งค่ากลับเป็น const char*
void	setServiceName (const char* n) ใช้ในการกำหนดชื่อของเซอร์วิสจากพารามิเตอร์ const char* n
const char*	getServiceName () const ใช้ในการเรียกดูชื่อของเซอร์วิส และส่งค่ากลับเป็น const char*
void	setServiceParameter(const char* p) ใช้ในการกำหนดค่าของเซอร์วิสพารามิเตอร์จาก const char* p
const char*	getServiceParameter () const ใช้เรียกดูค่าของเซอร์วิสพารามิเตอร์ และส่งค่า const char*
void	setError (const char* e) ใช้ในการกำหนดค่าของข้อความผิดพลาด ซึ่งกำหนดโดยพารามิเตอร์ const char* e ที่บอกถึงข้อความบอกความผิดพลาดที่ต้องการกำหนดให้กับแอตทริบิวต์ Str error ของคลาส
const char*	getError () const เป็นคอนสแตนต์ฟังก์ชันที่ใช้ในการเรียกดูข้อความบอกความผิดพลาดของคลาสที่มีอยู่ปัจจุบัน และส่งค่ากลับเป็น const char*

2.11 คลาสโซบพาร์เซอร์ (SOAPParser Class) เป็นคลาสที่ใช้ในการตรวจสอบโครงสร้างของเอกสารโซบว่าถูกต้องตามมาตรฐานของโซบ หรือไม่โดยจะมีการเรียกใช้งานจากคลาสเอ็กซ์เอ็มแอลพาร์เซอร์ (XMLParser Class) ซึ่งจะมีฟังก์ชันที่ใช้ในการตรวจสอบโครงสร้างตามมาตรฐานโซบ และฟังก์ชันที่ใช้ในการสร้างโซบออบเจกต์ (SOAP Object) จากโซบแมสเซจ

คอนสตรัคเตอร์ (Constructor)	
SOAPParser()	สร้างอินสแตนซ์เพื่อทำการกำหนดค่าภายหลังหรือเป็นดีฟอลท์คอนสตรัคเตอร์(Default Constructor)

ฟังก์ชัน (Function)

static Envelope*	parse(const char* message) ใช้ในการพาสโซบแมสเสจเพื่อตรวจสอบโครงสร้างและทำการสร้างอินสแตนซ์ของคลาส Envelope และส่งค่ากลับในรูปแบบของ Envelope*
static char*	getServiceName(const char *n) ใช้ในการเรียกค่าของชื่อเซอร์วิสที่ต้องการเรียกใช้ และส่งค่ากลับเป็น char*
static int	checkServiceName(const char *n) ใช้ในการตรวจสอบชื่อเซอร์วิสที่ทำการเรียกใช้
Void	checkSyntax () ใช้ในการตรวจสอบความถูกต้องของโครงสร้างของโซบแมสเสจ

2.12 คลาสโซบรีเควสแฮนเดิลเลอร์ (SOAPRequestHandler) เป็นคลาสที่ใช้ในการจัดการโซบแมสเสจ ที่เข้ามาเพื่อเรียกใช้บริการเว็บเซอร์วิส โดยจะมีฟังก์ชันที่ใช้ในการตรวจสอบรายชื่อ และเมธอดของบริการนั้นว่ามีภายในระบบหรือไม่ ฟังก์ชันที่ใช้ในการกำหนดส่วนกำหนดข้อผิดพลาดของโซบ (SOAP Fault) ฟังก์ชันที่เรียกใช้คลาสดีซีรีไรซ์เซอร์ เพื่อทำการแปลงค่าที่ถูกกำหนดรูปแบบภายในโซบแมสเสจตามข้อกำหนดในส่วนของการเข้ารหัสของโซบให้เป็นชนิดของข้อมูล และฟังก์ชันที่ใช้ในการเขียนข้อมูลนำเข้าไปยังไฟล์ และฟังก์ชันที่ใช้ในการตรวจสอบไวยากรณ์ของเอกสารว่าตรงกับมาตรฐานโซบ ฟังก์ชันที่ใช้ในการตรวจสอบรายการของพารามิเตอร์นำเข้าว่าตรงกับรายละเอียดของบริการหรือไม่

คอนสตรัคเตอร์ (Constructor)	
SOAPRequest()	สร้างอินสแตนซ์เพื่อทำการกำหนดค่าภายหลังหรือเป็นดีฟอลท์คอนสตรัคเตอร์(Default Constructor)

ฟังก์ชัน (Function)	
Envelope*	buildEnv() ใช้ในการสร้างอินสแตนซ์ของคลาส Envelope จากโซบแมสเสจที่ได้รับจากผู้ใช้
Fault	buildSOAPFault () ใช้ในการสร้างอินสแตนซ์ของคลาส Fault เพื่อบอกข้อผิดพลาดของโซบแมสเสจที่ได้รับจากผู้ใช้
void	checkError () ใช้ในการตรวจสอบความผิดพลาดที่เกิดขึ้นในแต่ละส่วนของอินสแตนซ์ของคลาสที่ได้จากการสร้างมาจากโซบแมสเสจ

2.13 คลาสโซบเรสปอนส์แฮนเดิลเลอร์ (SOAPReponseHandler Class) เป็นคลาสที่ใช้ในการสร้างโซบแมสเสจ เพื่อตอบรับการเรียกใช้บริการเว็บเซอร์วิส โดยจะมีฟังก์ชันที่ใช้ในการสร้างโซบแมสเสจของบริการนั้นโดยดูจากรายละเอียดของบริการนั้น ฟังก์ชันที่ใช้ในการกำหนดส่วนกำหนดข้อผิดพลาดของโซบ (SOAP Fault) ฟังก์ชันที่เรียกใช้คลาสดีซีรีไรซ์เซอร์ เพื่อทำการแปลงชนิดของ

ข้อมูลให้เป็นค่าที่ถูกกำหนดรูปแบบภายในโซบแมสเสจ ตามข้อกำหนดในส่วนของการเข้ารหัสของโซบ และฟังก์ชันที่ใช้ในการเขียนข้อมูลนำเข้าไปยังไฟล์ และฟังก์ชันที่ใช้ในการตรวจสอบไวยากรณ์ของเอกสารว่าตรงกับมาตรฐานโซบ ฟังก์ชันที่ใช้ในการตรวจสอบรายการของผลลัพธ์พารามิเตอร์ว่าตรงกับรายละเอียดของบริการหรือไม่

คอนสตรัคเตอร์ (Constructor)	
SOAPResponseHandler()	สร้างอินสแตนซ์เพื่อทำการกำหนดค่าภายหลังหรือเป็นดีฟอลท์คอนสตรัคเตอร์(Default Constructor)

ฟังก์ชัน (Function)	
static char*	buildResponse(Envelope *env, const char* param) ใช้ในการสร้างโซบแมสเสจตอบรับกลับไปให้กับผู้ใช้ โดยนำส่วนของ

2.14 คลาสโซบเซิร์ฟเวอร์ (SOAPServer Class) เป็นคลาสที่ใช้ในการให้บริการเว็บเซอร์วิส โดยจะทำหน้าที่ในการรับ SOAP Message เข้ามาแล้วเรียกใช้คลาสโซบรีเคิวแอสเตอร์เลออร์ และคลาสโซบเรสพอนแอสเตอร์เลออร์ เพื่อจัดการสร้างโซบออบเจกต์ และตรวจสอบความถูกต้องตามที่ได้กล่าวแล้ว และยังมีหน้าที่ในการเชื่อมต่อกับส่วนของเว็บเซิร์ฟเวอร์

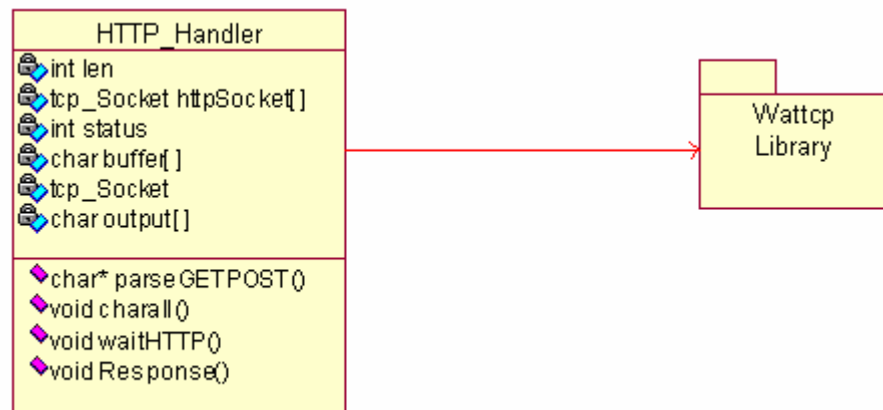
แอททริบิวต์ (Attribute)	
char	soapMess [] เก็บค่าโซบแมสเสจ
Envelope*	env เก็บพอยน์เตอร์ของคลาส Envelope ที่ได้รับมาจากผู้ใช้
Fault	error เก็บอินสแตนซ์ของคลาส Fault เพื่อบอกถึงข้อผิดพลาดที่อาจเกิดขึ้น

คอนสตรัคเตอร์ (Constructor)	
SOAPServer()	สร้างอินสแตนซ์เพื่อทำการกำหนดค่าภายหลังหรือเป็นดีฟอลท์คอนสตรัคเตอร์(Default Constructor)

ฟังก์ชัน (Function)	
Envelope*	recieveRequest () ใช้ในการสร้างอินสแตนซ์ของคลาส Envelope จากโซบแมสเสจที่ได้รับมาจากผู้ใช้และส่งกลับเป็น Envelope*
void	sendResponse () ใช้ในการเขียนโซบแมสเสจตอบรับให้ผู้ใช้ลงในเท็กซ์ไฟล์

3. เว็บเซิร์ฟเวอร์

ส่วนของเว็บเซิร์ฟเวอร์เป็นส่วนที่ใช้ในการจัดการข้อมูลส่วนหัวที่ตามโปรโตคอลเอชทีทีพีซึ่งจะทำการตรวจสอบข้อมูลและทำการพาสเอกสารในส่วนที่เป็นโซบแมสเสจโดยการเรียกใช้โซบพาร์เซอร์เพื่อทำการสร้างโซบแมสเสจที่ตอบกลับไปให้ผู้เรียกใช้เซอร์วิสโดยทำการเพิ่มข้อมูลส่วนหัวตามโปรโตคอลเอชทีทีพีและโซบ



รูปที่ 3-8 คลาสไดอะแกรมในส่วนของเว็บเซิร์ฟเวอร์

3.1 เอชทีทีพีแฮนด์เลอร์ (HTTPHandler Class) เป็นคลาสที่ใช้ในการจัดการข้อมูลในส่วนของโปรโตคอลเอชทีทีพีที่ได้รับเข้ามา แล้วทำการถอดเอาเฮดเดอร์และข้อมูลที่ต้องการออกมาและสร้างผลลัพธ์แล้วส่งกลับไปโดยจะทำการนำส่วนของวัตชีพีไลบรารีเข้ามาใช้เนื่องจากเป็นไลบรารีที่ใช้ในการติดต่อกับโปรโตคอลที่ซีพีของบอร์ดคอม 86

แอททริบิวต์ (Attribute)	
int	len เก็บขนาดความยาวของข้อมูล
tcp_Socket	httpSocket [] เป็นอินสแตนซ์ของคลาส tcp_Socket
int	status ใช้ในการบอกถึงสถานะการทำงานของคลาส
char	buffer[] เก็บข้อมูลที่รับเข้ามา
tcp_Socket	ใช้เป็นบัฟเฟอร์ในการติดต่อผ่านซ็อกเก็ต
char	output [] ใช้ในการเก็บข้อมูลที่จะตอบรับกลับ ไปให้กับผู้ใช้

คอนสตรัคเตอร์ (Constructor)	
HTTPHandler()	สร้างอินสแตนซ์เพื่อทำการกำหนดค่าภายหลังหรือเป็นดีฟอลท์คอนสตรัคเตอร์(Default Constructor)

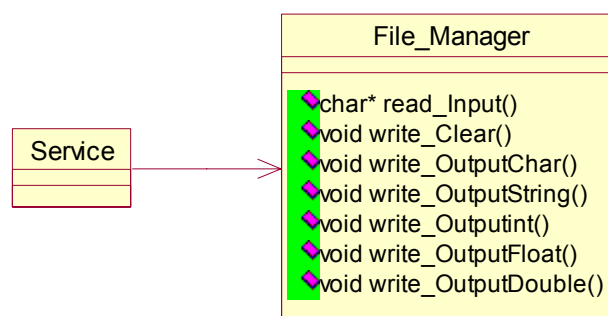
ฟังก์ชัน (Function)	
char*	parseGETPOST () ใช้ในการจัดการกับส่วนของเฮดเดอร์ในส่วนที่เกี่ยวกับโปรโตคอลเอชทีทีพี และสร้างเท็กส์ไฟล์ที่มีส่วนของโซบแมส

	เสร็จแล้วทำการเรียกใช้ส่วนของໂ໊ບພາຣ໌ເຊີ້ມາทำการจัดการໂ໊ບແມສ เสร็จต่อไป
void	clearAll () ใช้ในการกำหนดค่าภายในคลาสให้กลับเป็นค่าเริ่มต้น
void	waitHTTP () ใช้ในการรอรับการติดต่อจากผู้ใช้และสร้างการเชื่อม ระหว่างผู้ใช้กับเซิร์ฟเวอร์
void	Response () ใช้ในการสร้างເສດເດີຣ໌ในส่วน of ໂປຣໂຕຄອລເອັທີ່ຟີ และนำส่วนของໂ໊ບແມສເສດທີ່ตอบรับกลับไปให้กับผู้ใช้ที่ได้จากເຕັກ໌ ໄຟລ໌ที่สร้างจากໂ໊ບພາຣ໌ເຊີ້

3.2 ວັດທິຊີຟີໂລບຣາລີເປັນໂລບຣາລີທີ່ຈັດການກ່ຽວກັບການຕິດຕໍ່ສື່ສານຜ່ານທາງໂປຣໂຕຄອລຂອງບອຣ໌ດໂຄມ86
ຊຶ່ງເປັນຕົວຢ່າງແປດຟອມ໌ທີ່ເລືອກຊື້ນມາ

4. ອິນເຕີຣ໌ເຟສຂອງເຊີຣ໌ວິສ໌ໂຄດ໌

ເປັນສ່ວນທີ່ໃຫ້ເຊີຣ໌ວິສ໌ຕິດຕໍ່ກັບໄຟລ໌ເພື່ອໃຊ້ໃນການເຂົ້າຂໍ້ມູນທີ່ໄດ້ຈາກການທຳງານລຸ່ມເຕັກ໌ໄຟລ໌
ຕາມຮູບແບບເພື່ອໃຫ້ໂ໊ບພາຣ໌ເຊີ້ໃຊ້ໃນການສ້າງໂ໊ບແມສເສດຕອບກັບໄປຍັງໃຜູ້ໃຊ້



ຮູບທີ່ 3-9 ຄລາສໄດອະແກຣມໃນສ່ວນຂອງເຊີຣ໌ວິສ໌ໂຄດ໌

4.1 ຄລາສໄຟລ໌ມາເນຈເຣີ (FileManage Class) ເປັນຄລາສທີ່ໃຊ້ໃນການຕິດຕໍ່ກັບໄຟລ໌ໃນການເຂົ້າຂໍ້ມູນ
ຜລັກ໌ທີ່ຕ້ອງການຈະຕອບກັບໄປຢູ່ໃນຮູບຂອງເຕັກ໌ໄຟລ໌ເພື່ອໃຫ້ໂ໊ບພາຣ໌ເຊີ້ປ່ຽນໃຫ້ຢູ່ໃນຮູບແບບ
ມາຕາຮາຸນຂົມເປີ້ລອບເຈັດແອັດເຊສໂປຣໂຕຄອລຕໍ່ໄປ

ໂຄນສຕຣັກເຕີຣ໌ (Constructor)	
XMLReader ()	ສ້າງອິນສເຕັນຊ໌ເພື່ອທຳການກຳນົດຄ່າຢູ່ຫຼັງຫຼືເປັນດີຟອລ໌ທ໌ໂຄນສຕຣັກເຕີຣ໌ (Default Constructor)
ຟັງກ໌ຊັນ (Function)	
char*	read_Input () ໃຊ້ໃນການອ່ານຂໍ້ມູນທີ່ອ່ານໄດ້ຈາກສ່ວນຂອງໂ໊ບພາຣ໌ເຊີ້ ແລ້ວເກັບໄວ້ໃນຮູບຂອງເຕັກ໌ໄຟລ໌ ແລະສົ່ງຄ່າກັບໃນຮູບຂອງ char*

void	write_Clear () ใช้ในการลบข้อมูลในเท็กซ์ไฟล์เอาต์พุตที่จะใช้ในการเขียนข้อมูลที่ได้จากการทำงานของเซอร์วิสโค้ด
void	write_OutputChar () ใช้ในการเขียนข้อมูลชนิด char ลงในเท็กซ์ไฟล์เอาต์พุต
void	write_OutputString () ใช้ในการเขียนข้อมูลชนิด String หรือ char* ลงในเท็กซ์ไฟล์เอาต์พุต
void	write_Outputint () ใช้ในการเขียนข้อมูลชนิด int ลงในเท็กซ์ไฟล์เอาต์พุต
void	write_OutputFloat () ใช้ในการเขียนข้อมูลชนิด float ลงในเท็กซ์ไฟล์เอาต์พุต
void	write_OutputDouble () ใช้ในการเขียนข้อมูลชนิด double ลงในเท็กซ์ไฟล์เอาต์พุต

4.2 คลาสเซอร์วิสโค้ด (ServiceCode Class) เป็นคลาสที่ผู้ใช้ทำการสร้างขึ้นเพื่อสร้างเป็นเซอร์วิสโดยจะต้องทำการเรียกใช้คลาสไฟล์มาเนจเจอร์เพื่อเขียนข้อมูลผลลัพธ์ตามชนิดของข้อมูล โดยที่ส่วนประกอบส่วนอื่น ๆ ของเซอร์วิสนั้นผู้ใช้สามารถสร้างขึ้นได้เองตามที่ต้องการ